



 slington college
(इरिलिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Final Year Project

25% FYP Proposal

Semester

2024-25 Autumn

PROJECT TITLE: Sonic Points

Student Name: Sakchyam Thapa

London Met ID: 22067800

College ID: np01cp4a220185@islingtoncollege.edu.np

Internal Supervisor: Dipesh Raj Adhikari

External Supervisor: Abhishek Rokaya

Assignment Due Date: 2025/01/08

Assignment Submission Date: 2025/01/08

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded

ACKNOWLEDGEMENTS:

I am, indeed, grateful to the respected faculty of Islington for guiding and supporting me throughout my academic career. My research skills and technical knowledge have improved tremendously-all kudos to our college for building such an encouraging learning environment. Furthermore, I would also want to thank my supervisors,

Mr. Dipesh Raj Adhikari and **Mr. Abhishek Rokaya**, for their comprehensive feedback and mentorship. After all, it was their belief in my capabilities and timely advice that directed me through the obstacles and impelled this venture to success. Lastly, I am highly indebted to all the members of Islington College; guys, you all really motivated me, assisted me in areas whenever I needed it, which helped me a lot in my life.

ABSTRACT:

The "Sonic Points" project is a task management system that aims to increase productivity with gamification. It is integrated with user authentication, task tracking, and points allocation, as well as feedback, leaderboards, and rewards. Users receive points for completing tasks, affecting their position on the leaderboard. This will also allow managers to monitor performance, generate reports, and handle rewards. The system is supported by a normalized database structure for efficient and secure data storage. With Sonic Points, organizations can create and manage tasks; the.NET-, HTML-, CSS-, and JavaScript-developed system ensures that it will be engaging to employees and act like a feedback motivator to better productivity within organizations.

Contents

1. INTRODUCTION.....	6
1.1. Problem Scenario	8
1.2. Project as Solution.....	9
1.3. Aims and Objectives	10
1.3.1. Aims.....	10
1.3.2. Objectives	10
1.4. PROJECT DESCRIPTION	11
2. BACKGROUND:.....	13
2.1. UNDERSTANDING THE PROJECT SOLUTION'	13
2.2. EXISTING SYSTEM ANALYSIS:	15
2.3. Methodology:	24
2.3.1. Kanban Methodology:	24
2.4. Gantt Chart	25
2.5. Work breakdown structure:	26
3. WORK TO DATE:.....	28
3.1. SRS (SOFTWARE REQUIREMENTS SPECIFICATIONS):.....	28
3.1.1 PURPOSE:	28
3.1.2. REQUIREMENTS SPECIFICATIONS OF AN SRS:	28
3.1.2. DOCUMENT CONVENTIONS:	28
3.1.3. Functional Requirements	29
3.1.4. Non-Functional Requirements.....	32
3.1.5. REQUIREMENT GATHERING:	32
3.1.5.1. Literature Review: Task Management Systems and KPI-Driven Platforms.....	32
3.2. Design Specifications:	34
3.2.1. UI Design:	34
3.2.2. Backend Code	39
3.2.3. Database.....	45
3.3. Development Till Date:.....	48
3.3.1. Data Flow Diagram:.....	48
3.3.2. Use Case	50
3.3.3. High Level Use Case & Extended Use Case:	51
3.3.4. Sequence Diagram:	51

3.3.5.	Architecture Diagram:	56
3.3.6.	Class Diagram:	57
3.3.7.	Normilization	58
3.3.8.	Progress Table	58
4.	Further Development:	59
5.	References	62
6.	Apendix	63
6.1.	Hardware Requirements	63
6.2.	Software Requirements	63
6.3.	Key Dissimilarities of Sonic Points:	65
6.4.	REQUIREMENTS Gathering:.....	66
6.5.	Non Functional Requirements.....	69
6.6.	REQUIREMENTS SPECIFICATIONS OF AN SRS:	72
6.7.	High Level Use Case	73
6.8.	Extended Use	76
6.9.	Normalization.....	85
6.9.1.	Identify Repeating Groups	86
6.9.2.	NF (1NF) and Converting to 2NF:.....	86
6.9.3.	NF and Converting to 3NF	88
6.9.4.	Final Normalized Tables (3NF):	89
6.10.	Literature Review: Task Management Systems and KPI-Driven Platforms.....	89
6.11.	Feedback Form.....	92

1. INTRODUCTION

In the present world full of speed and target-based approaches, Key Performance Indicators (KPIs) have emerged as a fundamental tool for progressive, successful, and developing strategies. **KPIs** are measurable values that demonstrate how effectively an individual or organization is achieving key business objectives. **KPIs** constitute an essential system that encompasses principles of accountability, transparency, productivity,

KPI-driven solutions are becoming more important to organizations and individuals as they provide actual data to analyze efficiency, supervise task completion rates and optimize workflows. The reason for this can lie in the fact that KPIs not only reveal the current level of performance, but also help organizations predict the trends guiding their processes to overcome challenges and make decisions to meet deadlines and completely achieve their objectives; all with an efficient indicator. The increasing focus on data-driven approaches calls for delegation and productivity-enhancing tools that cover that space.

Welcome to **Sonic Points**, the task management program that can help you be the best version of yourself by incorporating modern KPI gear and game mechanics: By reaching towards speed, efficiency, and achievement, the name "**Sonic Points**" embodies users' opportunity to become more effective, as they complete tasks in real time and receive rewards instantly.

Through the following features, Sonic Points ensures both productivity acceleration and user-engagement-maintenance goals:

- ❖ **Sonic** — Speed: Platform encourages quick and efficient task completion in line with their Sonic personal
- ❖ **Points System**: Users are rewarded points for completing certain affirmations, and through this engagement cycle, they can redeem their earned points on meaningful and relatable rewards.

- ❖ **Leaderboard:** You can create a live leaderboard in your app, which greatly helps to instill healthy competition among users, who will work together as a community and get more done.

With Sonic Points, every quest you pursue contributes to your overall success.

1.1. Problem Scenario

Usually expected to balance multiple projects, deadlines and priorities, only to be faced with the normal problems of:

- ✚ **Task Disorganization:** Tasks are in various tools and platforms thus it is difficult to have a holistic view.
- ✚ **Progress Tracking Challenge:** The users were sceptical to track a number of tasks going through completion and in parallel, which led to many tasks going beyond their expected deadline.
- ✚ **Lack of a central collaboration hub:** Teams lack a centralized platform for assigning tasks, tracking progress, and communicating.
- ✚ **Limited Transparency** – The tasks or projects are barely visible so it is far harder to plan and allocate resources.
- ✚ **Lack of Motivation:** Many users face procrastination or feel uninterested, which interrupts their productive state.

Such problems result in inefficiency, delay and miscommunication

1.2. Project as Solution

It is aimed at overcoming common usages of project management systems such as centralized management of these affairs, prioritization of tasks and collaboration. Key features include:

- ✚ Multinomial Task Management: Organize, prioritize, and handle your tasks on one organizers.
- ✚ Ask & Discuss: Team members can easily answer and discuss questions.
- ✚ There is a lot of things to be mentioned about task prioritization and categorization: You need to do what is important first.
- ✚ Progress Tracking: Keep track of status of task with visual status indicators like, progress bars and checklists.
- ✚ Projects Manage: Arrange as project and Monitor aggregate status.
- ✚ User Roles & Permissions: Admins & managers manage task assignment & tracking.
- ✚ Gamification: Making points, running reward and leaderboards to incentivize users
- ✚ KPI System — Keep track of how your team and each person individually are doing with customizable KPIs and suggestions to improve.

1.3. Aims and Objectives

1.3.1. Aims

The Task Management tool is meant to organize such tasks and help the productivity of all users with its task tracking, gamified task management, peak flow, microlist & KPIs insight to push collaboration and increase user activity.

1.3.2. Objectives

- Login: Security login/ register + role based access (Admin, Manager, Team Member) ➤ There is a simple one which enables you to create, update, delete, set priorities (low, medium, high) and categorize (personal, work, urgent, etc.) tasks.
- Gamification: establish points-based rewards program (coffee, pens, etc.) and leaderboard for some light-play competition
- Performance of Tasks: A KPI also needs to be from where one should ideally finish the.
- Task and KPI the konko with timeline, this can be in the form of dashboard with holistic view to visualize insight and improvement

1.4. PROJECT DESCRIPTION

The Sonic Points project is an integrated task management system, gamifying the way work is done for better engagement of workers and ultimately increased productivity. The system uses a point-based reward system and key performance indicators to make the worker keep on pressing for the completion of the task efficiently and in time. Also, user authentication, task management, point allocation, leaderboard, reward management, feedback, and report generation features are integrated into the system to turn the work environment into an arena of interactivity and competitiveness. Features: User Authentication: Employees can register and log in with secured credentials provided. Only the admin will be able to verify and manage the data of users, ensuring proper access. Task Management: Employees may create, manage, and track tasks with a deadline. Also, the real-time update of the task progress makes it possible for users to update when the task is accomplished. Point Allocation and Leaderboards: Employees will receive points on task completion, and the points will be updated in the system. In addition, the best-scoring employees will be updated dynamically on the leaderboard for further peer-to-peer competition.

Key Features:

User Authentication:

- Employees can sign up and log in with safe credentials.
- Admins verify and control users' information to guarantee proper access.

Task Management:

- Workers will be able to create, manage, and track tasks due on a certain date.
- Tasks progress gets updated automatically in real time; hence, users are allowed to mark tasks as completed.

Points Allocation and Leaderboards:

- Employees get points for the completion of tasks, and the system updates them.
- The leaderboards show an overview of top performers, promoting healthy competition among workers.

Reward Management:

- Points can be redeemed by employees for rewards, hence motivating them.

Administrative Panel: Offers a reward option and makes the distribution evenly

Feedback System:

- Employees may share feedback on task and work experience.
- Reviewing and reports can be formed by managers over the feedback generated to inspect sentiments and engagements with employees.

Report Generation:

- Formation of detailed reports is possible over individual activities—performance, tasks accomplishment, and the redemption of points by employees.

Other Functionalities

Real-time Alerts: Notifications for changed tasks, assigned points, and leaderboard shift live.

Access Base: Separate level of access afforded to an employee, manager, and then administrative access.

Easy Operability: Effortless interfacing allows ultimate comfort in UI.

2. BACKGROUND:

The **Sonic Points** project tries to solve the problem of task management and employee engagement in organizations through the implementation of a gamified points system. Designed to increase productivity, the system integrates key performance indicators, task tracking, and a reward mechanism that encourages employees to complete tasks in a more effective way. Office management teams are the target users of the system, and it provides value both for employees and managers. Employees can hence view tasks, track their progress, and get rewards based on performance, while managers will view detailed reports, the leaderboard, and employee feedback.

Most of the task management systems currently in existence are not fully equipped to motivate or engage employees. Traditional systems normally just record task completion and offer no incentives for performance, nor is there any tracking of an employee's performance over time. That is where Sonic Points fills the gap by implementing a points system to get employees being proactive and completing tasks, striving for higher performance. Besides, the introduction of a leaderboard and rewards system will motivate them via healthy competition.

The features make this project unique in the traditional task management system, such as innovatively introducing employee productivity, increasing the rate of completion of tasks, and giving managers insights with which to make data-driven decisions. The Sonic Points system, user-centric in design, thus creates an environment where employees and managers alike can track progress toward achieving set goals at higher levels of engagement and efficiency.

2.1. UNDERSTANDING THE PROJECT SOLUTION'

A deep study of the present system of task management and feedback highlighted some of the key issues with regard to engagement into tasks, motivations for task completions, and tracking performance. The key features necessary for the new system that have been integrated into Sonic Points include rewards for completing a task and features spurring competition. This will involve rewarding staff with certain points upon completion of a task, thereby developing a real-time leader board displaying friendly team competition. The key takeaways from the analysis and feedback include the following:

1. **Need for Gamification:** There was a need expressed by the users for a system that could make task management more interactive and fun. Integration of points and leaderboards addresses this need, hence giving employees tangible reasons to perform better.
2. **Performance Tracking:** There was an interest on the part of both employees and managers in tracking completed tasks and, generally, their performance. Among others, the system of Sonic Points allows managers to see detailed reports, track employee progress, and recognize the best performers.

3. Reward System: The other big flaw in most traditional systems is a lack of rewards or incentives. That is where this system differs, since it lets one stay atop their motivation in work by offering redeeming of points for some great rewards.

4. Feedback Management: Most of the time, employees feel disconnected with the management, and the feedback systems are not quite functional. The Sonic Points system lets employees provide feedback on tasks, enabling managers to gauge employee morale and where things may be going wrong.

5. Collaboration and Task Transparency: The system is very transparent and collaborative; the employee sees the status of the tasks and competes in a healthy manner, helping to improve communication and teamwork, hence a more efficient workplace.

6. Many different participants took part in the survey, including supervisors. and managers, employees, and freelancers.

7. The response from approximately 20 participants was of crucial importance to understand their experience and expectations.

8. The question form is based on user satisfaction, system performance and request features in the new system.

[Access the Survey Questions](#)

2.2. EXISTING SYSTEM ANALYSIS:

1. Trello: Task Management System

Trello is one of the more famous task management tools that help the user organize his tasks/projects into boards, lists, and cards. It boasts a drag-and-drop interface and is pretty popular among teams. Technologies Used:

- Frontend: React.js, HTML, CSS
- Backend: Node.js, MongoDB

Key Features:

- o Visual boards for task organization
- o Collaboration tools, eg team boards
- o Due dates, checklists, and attachments
- o Basic analytics for task tracking, though not KPI-driven

Comparison:

- Similarities:
 - o Both systems focus on task management and provide a visual representation of tasks and progress.
 - o Trello and Sonic Points allow users to track project progress, although Sonic Points incorporates KPIs and gamification, which Trello lacks.
- Differences:
 - o Sonic Points offers a leaderboard and points system for completing tasks, which adds a competitive, gamified element that Trello lacks.
 - o Sonic Points includes a KPI-driven approach that is focused on individual progress and performance metrics, whereas Trello is more collaborative and focuses on task organization rather than individual achievement.

TrelloWorkspaces

Jump to...

BoardMarketing: Product LaunchTravidux, LLCWorkspace visible

To Do

Phase 2

Design & Order Swag

Jun 8 - Jun 24

Phase 3

Social Media Content

Jun 15 2 0/5

Phase 2

Promo Video

Jul 6

Phase 3

Upload Tutorial Videos

Jun 10

+ Add another card

In Progress

Phase 3

Android Release Notes

Jun 10 1

Phase 2

PR

Jun 14 1 2/3

Phase 1

Update Logo

Jun 14 - Jun 30 1/3 Jun 9

Phase 3

iOS Release Notes

Jun 18

+ Add another card

Done

Phase 1

GTM Plan and Messaging

May 27 4/4

Phase 1

Localization/Translations

May 26

Phase 1

Design Assets - Illustrations and Graphics

May 4 1

+ Add another card



2. Todoist: Task Management with Productivity Tools

Overview: Todoist is a task management tool that provides a clean, simple interface to organize tasks and projects, and includes productivity-enhancing features such as labels, filters, and project tracking.

Technologies Used:

- Frontend: React.js, HTML, CSS
- Backend: Python, PostgreSQL
- Key Features:
 - Task categorization with labels, filters, and projects
 - Task priority levels and due dates
 - Basic reports and analytics on task completion (without gamification)

Comparison:

- Similarities:
 - Both Todoist and Sonic Points focus on helping users manage tasks efficiently.
 - Sonic Points and Todoist have a system for tracking task completion and progress.
- Differences:
 - Sonic Points integrates a points-based reward system, motivating users through gamification, while Todoist lacks a system for rewarding users.
 - Sonic Points also includes a leaderboard for competition, making it unique compared to Todoist, which is more focused on individual productivity and does not feature social competition.
 - Sonic Points incorporates KPI-driven solutions, offering insights into user performance, while Todoist primarily focuses on task organization and basic productivity metrics.



3. Asana: Team Collaboration and Task Management

Overview: Asana is a powerful task management tool used for team collaboration, project planning, and tracking. It is highly customizable, allowing teams to manage tasks, deadlines, and workflows effectively.

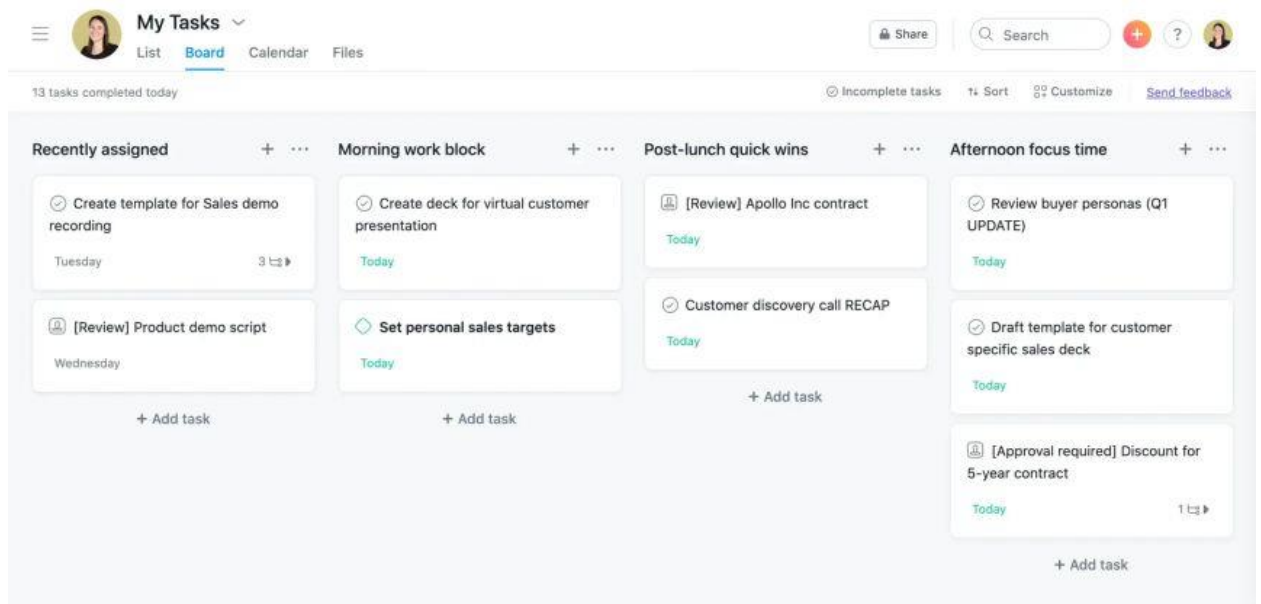
Technologies Used:

- Frontend: React.js, HTML, CSS
- Backend: Node.js, PostgreSQL
- Key Features:
 - Project and task tracking
 - Team collaboration and assignment
 - Timeline and calendar views
 - Reporting tools and integration with other software (e.g., Slack, Google Drive)

Comparison:

- Similarities:

- Both Asana and Sonic Points aim to enhance productivity through task management, supporting team collaboration (in Asana's case) and personal goal setting (in Sonic Points).
- Both platforms allow users to track the progress of tasks and projects.
- Differences:
 - Sonic Points stands out by integrating gamification through a leaderboard and a point system, which is not present in Asana.
 - Asana emphasizes team collaboration and project workflows, while Sonic Points is more focused on individual performance and KPIs, encouraging users to measure their progress against personal goals.
 - The KPI-driven focus in Sonic Points enables users to track performance and predict trends, which goes beyond Asana's basic project management functionalities.



4. Jira: Advanced Project and Task Management

Overview: Jira is a widely used project and issue tracking software designed to help teams plan, track, and release software projects. It allows users to manage tasks, projects, and workflows with a high degree of customization. Jira is particularly popular in agile development environments, enabling teams to manage tasks through customizable workflows, scrum boards, and Kanban boards.

Technologies Used:

- **Frontend:** React, JavaScript, HTML, CSS
- **Backend:** Java, PostgreSQL, MySQL
- **Key Features:**
 - Task management through customizable workflows
 - Agile tools like Scrum and Kanban boards
 - Reporting and analytics features for tracking project progress
 - Integration with other tools like Confluence, Bitbucket, and Trello
 - Customizable permissions and security settings

Comparison:

- **Similarities:**
 - Both Jira and Sonic Points focus on efficient task management and progress tracking, with features that support team collaboration.
 - Both platforms offer customizable workflows to fit different project needs, ensuring flexibility for users.
- **Differences:**
 - **Sonic Points** emphasizes gamification and performance tracking through KPIs, motivating employees to achieve specific goals. In contrast, Jira is more focused on traditional project management features like task tracking, issue resolution, and agile methodologies.
 - **Sonic Points** uses a points and rewards system to increase user engagement, while Jira focuses on workflow management, sprint planning, and project tracking without gamification.

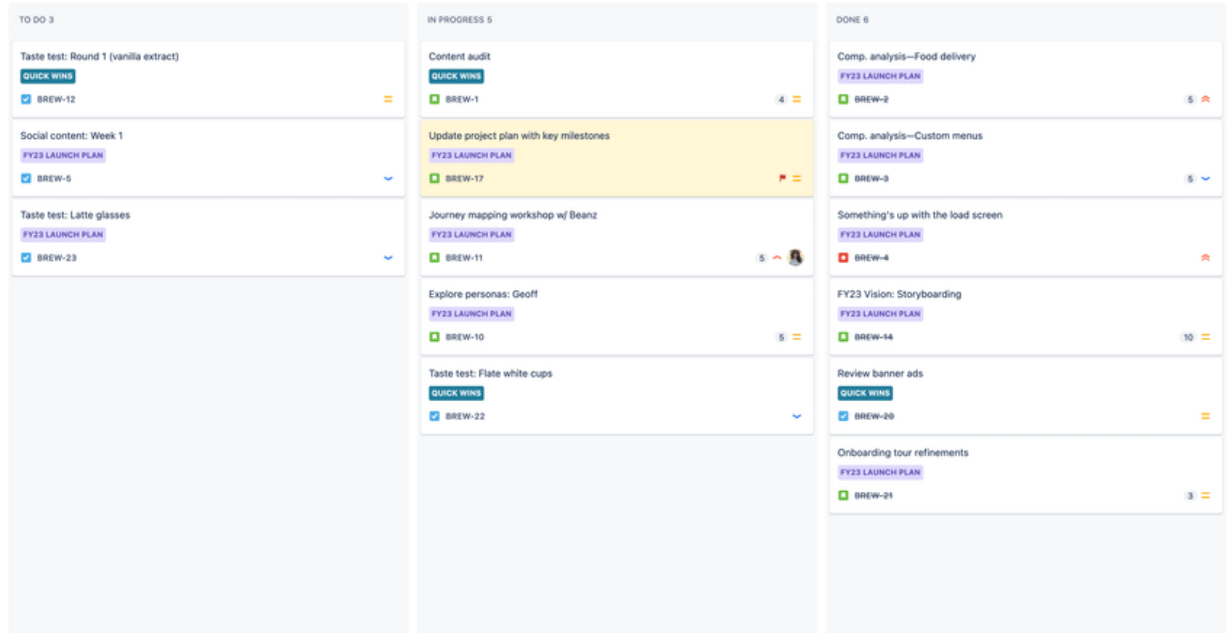
Affogato

As a user, I'm able to jump onto the Espresso app, order a brew and a snack for now, and pre-order dessert for later.

⚡ ☆ ⌚ 0 days remaining < Complete sprint ...

 Q Epic ▾ Type ▾ Quick filters ▾

Insights View settings



Feature	Sonic Points	Trello	Todoist	Asana	Jira
Gamification	Strong gamification with leaderboards, points, and rewards for task completion	No gamification, focuses on task management and organization	No gamification; focuses on task management and project tracking	No gamification; focuses on project and task management	Minimal gamification (e.g., story points for tasks), primarily focuses on agile project management
KPI-Driven Approach	Focuses on KPI tracking, offering insights into performance, trends, and progress predictions	Does not provide KPI tracking; focuses more on task organization and collaboration	No KPI-based performance insights, only task management	Lacks KPI-driven insights; focuses on team collaboration and project management	Offers limited KPI tracking via custom reports and third-party integrations; focuses more on agile metrics like velocity
Individual vs. Team Focus	Primarily focuses on individual productivity and	Primarily focuses on team	Primarily individual task	Primarily focused on team	Primarily team-focused, tailored for agile

	performance tracking	collaboration with shared boards and tasks	management with minimal team-oriented features	collaboration and project timelines	workflows and software development teams
Progress Tracking & Analytics	Offers detailed progress tracking with real-time insights, and trends to optimize workflows	Basic progress tracking (completed tasks) without detailed analytics	Provides progress tracking but lacks detailed performance analytics	Provides project and task completion tracking, but lacks KPI-driven analytics	Offers robust progress tracking for teams, including sprint metrics, burndown charts, and backlog tracking
Competition & Engagement	Includes a live leaderboard to foster healthy competition among users	No competition or engagement features beyond collaborative boards	No competition or leaderboard system for user engagement	No competitive features; focuses on team collaboration	No competitive features; focuses on team efficiency and task prioritization
Task Completion Rewards	Users receive points and rewards for completing tasks, redeemable for meaningful incentives	No reward system for task completion	No point system or rewards for task completion	No reward system for task completion	No reward system for task completion
Focus on Data-Driven Solutions	Uses real-time data to optimize productivity and task completion, offering actionable insights	Lacks a data-driven approach; focuses more on visual task organization	Lacks advanced data-driven features; focuses on task management	Focuses on project tracking but lacks advanced data-driven features	Strong focus on agile metrics and reporting, but primarily designed for software development teams
Task Management View	Primarily a single-user interface focused on individual goals and tasks	Primarily a team-based project management interface	Primarily designed for individual use, with team collaboration features	Focuses on team-based project management, tracking timelines and milestones	Team-focused view with agile boards (scrum/kanban), epics, and sprint planning

Key Dissimilarities of Sonic Points:

VIEW

Feature	Sonic Points	Trello	ToDoList	Asana	Jira
Gamification	✓	X	X	X	X
KPI-Driven Approach	✓	X	X	X	X
Individual vs. Team Focus	✓ (Individual)	✓ (Team)	✓ (Individual)	✓ (Team)	✓ (Team)
Progress Tracking & Analytics	✓	X	X	X	X
Competition & Engagement	✓	X	X	X	X
Task Completion Rewards	✓	X	X	X	X
Focus on Data-Driven Solutions	✓	X	X	X	X
Task Management View	✓ (Individual)	✓ (Team)	✓ (Individual)	✓ (Team)	✓ (Team)

2.3. Methodology:

2.3.1. Kanban Methodology:

The Kanban methodology will be utilized in the development of the Task Management Application emphasizing flexibility, continuous delivery and visual management. These tasks will move through stages based on their capacity

- ❖ Requirement Analysis

- ❖ Objective: To be able to understand user requirements and needs in order to ensure the services provided by the application are adequate.

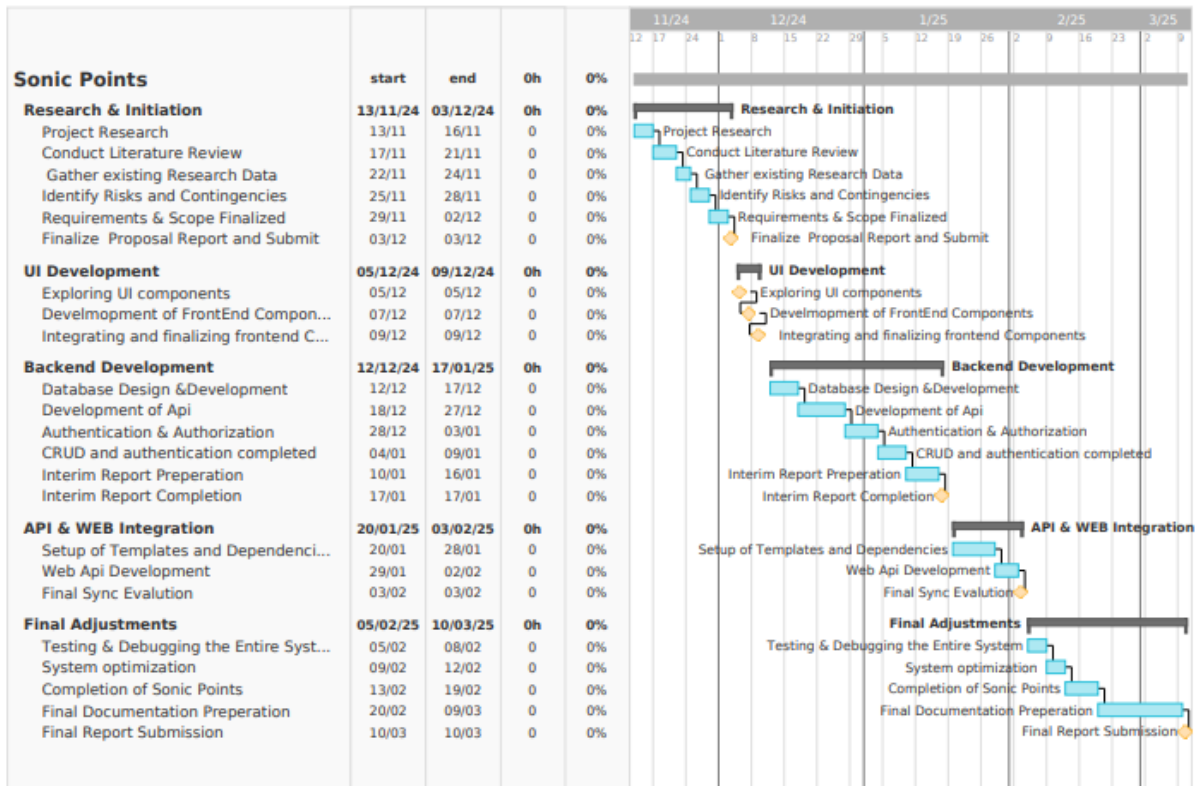
- ❖ Activities:

- o Generate leads through an interview and survey with potential users to discover their pain points and sort out the features they need.

- o Place these tasks in the Kanban board for maximum visibility and adherence to progress.

Selected Methodology and Requirement Analysis for my project are provided below in appendix section: [Link to appendix](#)

2.4. Gantt Chart



2.5. Work breakdown structure:

Same as initially planned for WBS because all the activities and deliverables that were planned up to the milestone of interim submission have been realized. Briefly, they include:

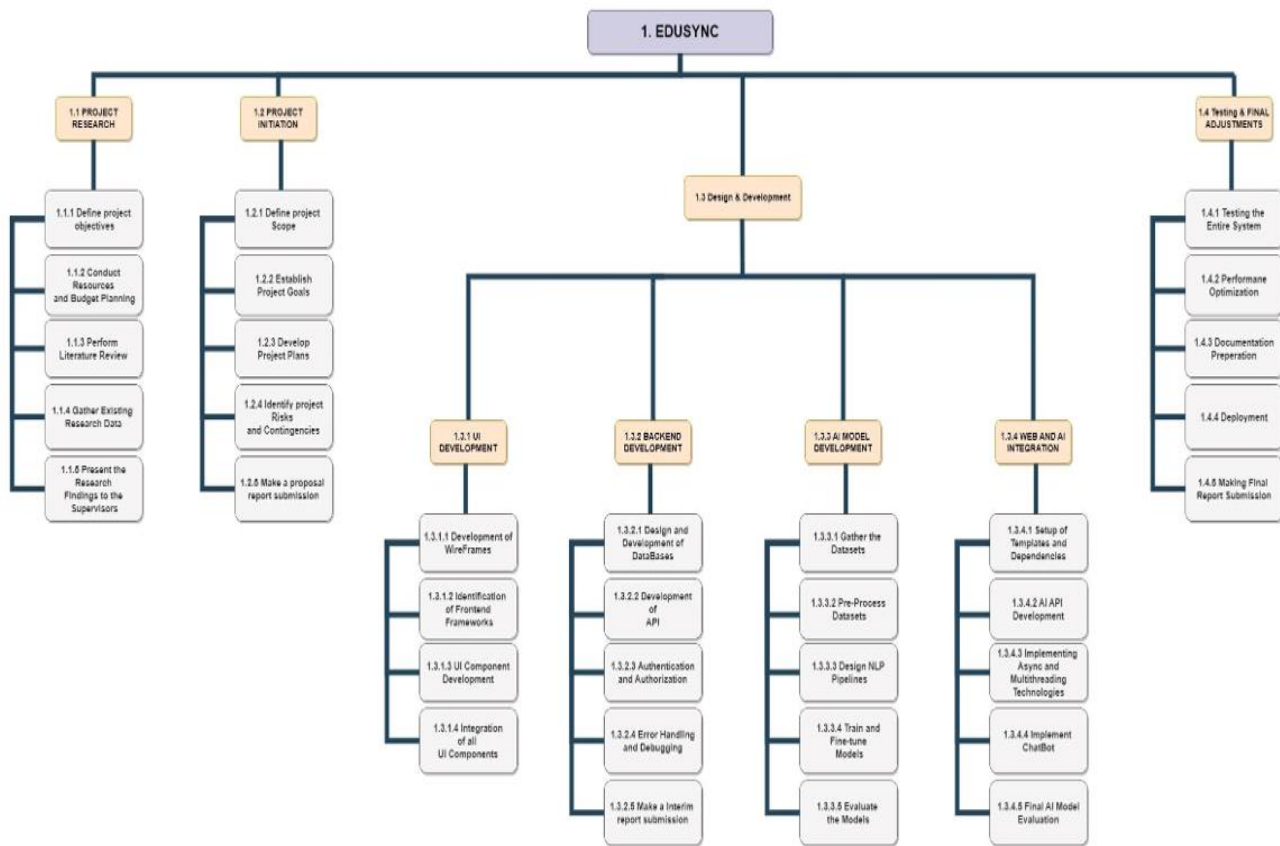
Initial Research and Planning

- ❖ Clearly define project goals
- ❖ Plan and budget resources
- ❖ Explore relevant technologies and similar systems

Interim Development Phase

- ❖ Prototyping and wire-framing for UI/UX design
- ❖ Front-end framework and tools selection
- ❖ Development of UI modular component to develop interface.
- ❖ Normalization of the database and preparation of the ERD
- ❖ SRS documentation
- ❖ Preparing the UML diagrams for the system

Since all tasks planned till this phase have been executed as scheduled, the WBS shall be updated only after identifying the next set of deliverables beyond the interim submission phase so that the work completed thus far aligns with the project plan and Any changes post that can be incorporated in a structured manner.



3. WORK TO DATE:

3.1. SRS (SOFTWARE REQUIREMENTS SPECIFICATIONS):

(VIEW)

3.1.1 PURPOSE:

This Software Requirement Document describes the Sonic Points System, a task management system designed to increase productivity and engagement through gamification and KPI-driven performance tracking. Generally, the system will provide ways for users to manage their tasks, track progress toward goals, and ultimately accomplish those goals in the most efficient manner while fostering healthy competition and collaboration.

Core features of the system are task creation and tracking, point allocation, leaderboards, and reward redemption. Sonic Points will combine performance metrics with gamification elements to enhance users' motivation and improve the speed of task completion by making the work environment enjoyable. It shall assist employees and managers in achieving better organizational efficiency, transparency, and accountability.

3.1.2. REQUIREMENTS SPECIFICATIONS OF AN SRS:

(View)

3.1.2. DOCUMENT CONVENTIONS:

Font Style and Size: Arial, size 12 for the body, bold for headings.

Requirement Priorities: The requirements will be set as High, Medium, or Low in terms of priority.

Terms: "Manager" is administered for admins, and "Employee" is used for regular users.

3.1.3. Functional Requirements

Functional requirements describe the specific behaviors, functions, and features of the system. These are the actions the system must perform to meet the needs of its users. Below is the detailed list of functional requirements for the "**Sonic Points**" project:

1. User Registration and Authentication

- **Description:** The system should allow users to register, log in, and manage their profiles.
- **Features:**
 - Users can create accounts with unique credentials (email and password).
 - Implement secure login/logout functionality.
 - Enable password recovery or reset mechanisms.
 - Option for OAuth integration (Google, Facebook, etc.) for easy registration/login.

2. Task Creation and Management

- **Description:** The system should allow users to create, edit, view, and delete tasks.
- **Features:**
 - Users can define tasks with attributes such as title, description, priority, and deadlines.
 - Tasks should be categorized (e.g., personal, professional).
 - Users can mark tasks as completed or in-progress.
 - Include reminders or notifications for upcoming deadlines.

3. KPI Tracking and Analysis

- **Description:** The system should track KPIs for individual users based on their task completion and provide insights.
- **Features:**
 - Track metrics like task completion rate, average time per task, and missed deadlines.
 - Display progress in a graphical or tabular format (e.g., charts or progress bars).
 - Allow users to filter data by date range or category.
 - Generate periodic performance reports (weekly/monthly).

4. Gamification Features

- **Description:** Integrate game mechanics to encourage user engagement and productivity.
 - **Features:**
 - Assign points for each completed task based on its complexity or priority.
 - Display real-time point balances to users.
 - Allow users to redeem points for virtual or tangible rewards.
 - Implement badges or achievements for milestone completions (e.g., "100 Tasks Completed").

5. Leaderboard Functionality

- **Description:** The system should foster healthy competition by ranking users based on their performance.
- **Features:**
 - Display a real-time leaderboard showing top performers within a community or team.
 - Leaderboard rankings should be based on points earned.
 - Include filters for viewing global, team-based, or individual leaderboards.
 - Highlight user achievements on the leaderboard (e.g., "Task Streak").

6. Dashboard Overview

- **Description:** Provide a centralized dashboard for users to view their progress and activities.
- **Features:**
 - Show a summary of tasks, points, leaderboard position, and KPI performance.
 - Include customizable widgets for a personalized user experience.
 - Quick links for frequently accessed sections like "Create Task" or "Redeem Points."

7. Rewards System

- **Description:** Users should be able to redeem earned points for rewards.
- **Features:**
 - Create a virtual store for users to browse available rewards.

- Deduct points from the user's balance upon redemption.
- Notify users when rewards are successfully redeemed.
- Provide an option to view reward history.

8. Notifications System

- **Description:** Notify users about important updates and reminders.
- **Features:**
 - Notify users about upcoming deadlines, new tasks, and leaderboard changes.
 - Include in-app notifications, email notifications, or SMS (if applicable).
 - Allow users to customize notification preferences.

9. Multi-Device Support

- **Description:** The platform should be accessible on various devices and screen sizes.
- **Features:**
 - Implement a responsive web design using HTML, CSS, and JavaScript.
 - Optimize for desktop, tablet, and mobile views.
 - Ensure seamless transitions and consistent UI/UX across devices.

10. Admin Panel

- **Description:** Provide administrators with tools to manage the system.
- **Features:**
 - Manage users (e.g., ban accounts, reset passwords).
 - Monitor leaderboard activities to prevent misuse (e.g., fraudulent point systems).
 - View and manage tasks, KPIs, and reward systems.
 - Generate analytical reports for overall system performance.

3.1.4. Non-Functional Requirements

Non-functional requirements define the attributes or qualities the system should possess. They are typically related to system performance, security, usability, and other operational aspects that are critical for the success of the system. Below are the non-functional requirements for the **"Sonic Points"** project:

Appendix: Nonfunctional requirements

3.1.5. REQUIREMENT GATHERING:

(VIEW)

3.1.5.1. Literature Review: Task Management Systems and KPI-Driven Platforms

The development of task management systems has been greatly influenced by the incorporation of Key Performance Indicators (KPIs) and gamification techniques. These systems are designed to improve productivity, engagement, and accountability, as demonstrated in various projects. Below is a review of similar projects, their development methodologies, and a comparison to your Sonic Points project, highlighting both similarities and differences.

1. "Research and Analysis of the Frontend Development Technologies"

- Summary: This paper discusses the essential role of HTML, CSS, and JavaScript in front-end development, emphasizing their combined use to create interactive and visually appealing web pages.
- Link: <https://www.ijeast.com/papers/47-52%2C%20Tesma611%2CIJEAST.pdf>

2. "Performance Analysis of Web Application using Microsoft .Net Technology"

- Summary: This study investigates the performance aspects of web applications developed using the Microsoft .NET Framework, highlighting its efficiency and scalability for back-end development.
- Link: https://www.researchgate.net/publication/284423419_Performance_Analysis_of_Web_Application_using_Microsoft_Net_Technology

3. "Analyzing Front-end Web Development Technologies Based on HTML, CSS, and JavaScript"

- Summary: This article provides an in-depth analysis of HTML, CSS, and JavaScript, focusing on their roles in creating responsive and user-friendly web interfaces.
- Link: <https://ijrpr.com/uploads/V5ISSUE3/IJRPR23526.pdf>

4. "Exploration of Web Front-end Development Technology and Optimization Direction"

- Summary: This paper explores various front-end development technologies, including HTML, CSS, and JavaScript, and discusses optimization strategies to enhance web performance.
- Link: https://www.researchgate.net/publication/308627965_Exploration_of_Web_frontend_development_technology_and_optimization_direction

5. "Web Development via HTML, CSS, and JavaScript"

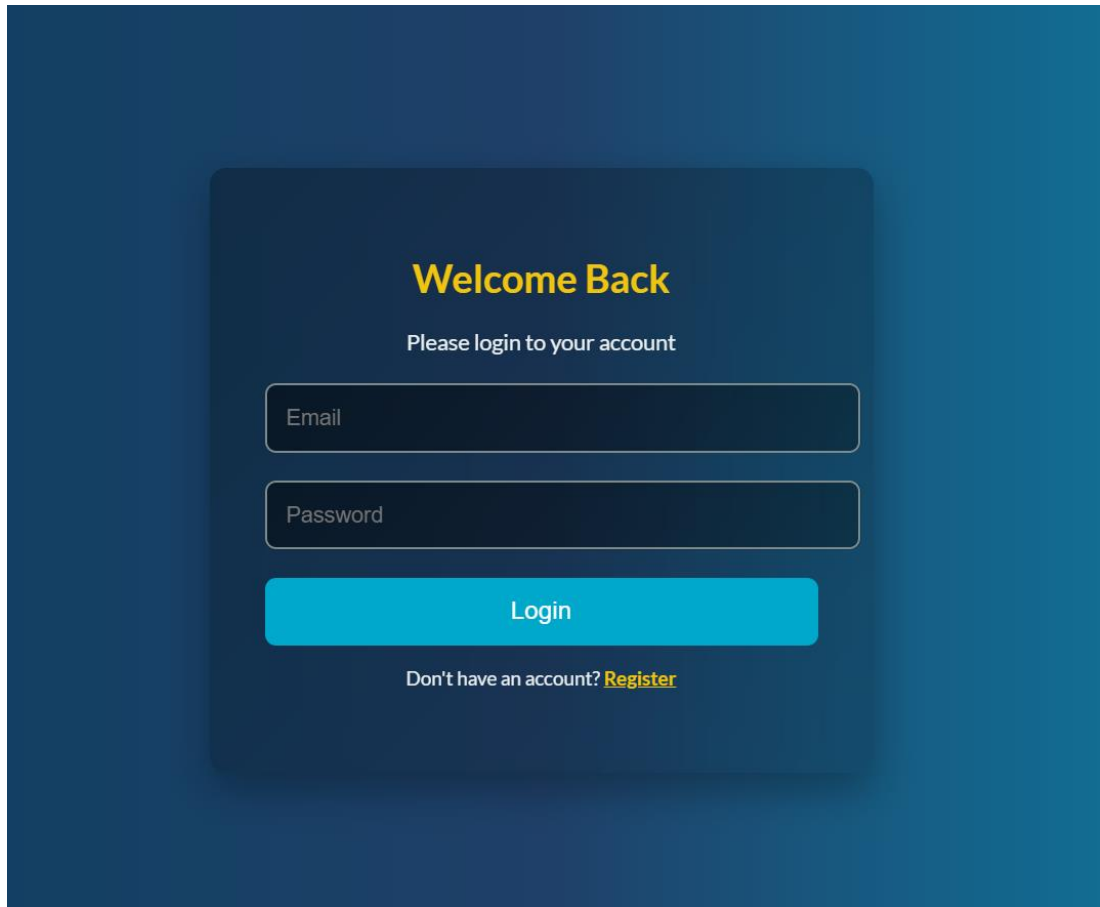
- Summary: This study examines the use of HTML, CSS, and JavaScript in web development, providing insights into their practical applications and benefits.
- Link: <https://ijrpr.com/uploads/V5ISSUE4/IJRPR25055.pdf>

6. "Understanding Web Front-end Development Technology"

- Summary: This paper discusses the key technologies used in web front-end development, including HTML, CSS, and JavaScript, and their impact on user experience.
- Link: <https://www.irjet.net/archives/V8/i6/IRJET-V8I6597.pdf>

3.2. Design Specifications:

3.2.1. UI Design:



Create Account

Register

Already have an account? [Login](#)

☰ Work Management

Add Task

☰ Backlog

Task 1

Task 2

🔄 In Progress

Task 3

🔍 Review

Task 5

✅ Completed

Task 4

Dashboard

Home / Dashboard

👤 Users and KPI

User	Email	KPI Points	Actions
John Doe	john.doe@example.com	85	🔄 Reset KPI
Jane Smith	jane.smith@example.com	90	🔄 Reset KPI

📁 Add New Project

Project Name

Deadline

mm / dd / yyyy

Description

Add Project



Redeem Your Sonic Points

★ 1,200 Points



Route 44 Drink

500 points

Redeem



Tater Tots

700 points

Redeem



Cheeseburger

800 points

Redeem

Previous

Next

FEEDBACK

Tell us more

☐ Complaint

☐ Suggestion

☒ Other

Rate Us:

 rating meh

 rating smile

 rating happy

Submit



Sonic Points Leaderboard

★ Your Points: 1200

Top Players

Rank	Name	Sonic Points
1	SonicFan99	1500
2	Speedster	1400
3	RingCollector	1350
4	BlazeRunner	1300
5	TailsBuddy	1250
6	KnucklesPunch	1200
7	ShadowX	1150

Welcome, Sarah Johnson!

Your Project Dashboard

Project Alpha

📅 Deadline: Dec 31, 2024



[View Details](#)

Project Beta

📅 Deadline: Jan 15, 2025



[View Details](#)

3.2.2. Backend Code

```
using Microsoft.AspNetCore.Identity;
using System.ComponentModel.DataAnnotations;

8 references
public class LoginHistory
{
    2 references
    public int Id { get; set; }
    6 references
    public string UserId { get; set; }
    6 references
    public string Username { get; set; }
    6 references
    public string Email { get; set; }
    6 references
    public DateTime LoginDateTime { get; set; }
    6 references
    public string Remarks { get; set; }
    6 references
    public string IP { get; set; }
    6 references
    public string UserActivity { get; set; }
}
```

```

[Route("api/[controller]")]
[ApiController]

public class ProjectController : _AbsAuthController<ProjectController>
{
    private readonly IProjectRepository _projectRepository;
    private readonly IVW_ProjectStatRepo _MainRepo;
    private readonly AppDbContext _context;

    public ProjectController(IProjectRepository projectRepository
        , IVW_ProjectStatRepo MainRepo
        , ILogger<ProjectController> logger
        , AppDbContext appDbContext)
        : base(logger)
    {
        _projectRepository = projectRepository;
        _context = appDbContext;
        _MainRepo = MainRepo;
    }

    [HttpGet]

    public async Task<ActionResult<IEnumerable<object>>> GetAllProjects()
    {
        try
        {
            // Get all vendors from repository
            List<VW_Projects_WithStat> projects = await _MainRepo.GetList()
                .ToListAsync();

            return Ok(new
            {
                success = true,
                message = "Projects retrieved successfully.",
                data = projects
            });
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "An error occurred while fetching all projects.");
            return StatusCode(500, new { success = false, message = $"Internal server error: {ex.Message}" });
        }
    }
}

```

```

using Microsoft.AspNetCore.Mvc;
using FileUploadAPI.Repositories;
using Microsoft.EntityFrameworkCore;
using FileUploadAPI.Models;
using FileUploadAPI.Data;
using FileUploadAPI.Dto;

namespace FileUploadAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]

    public class LoginHistoryController : _AbsAuthController<LoginHistoryController>
    {
        private readonly ILoginHistoryRepository _MainRepo;
        private readonly AppDbContext _context;

        public LoginHistoryController(ILoginHistoryRepository loginHistoryRepository, AppDbContext context, ILogger<LoginHis
            : base(logger)
        {
            _MainRepo = loginHistoryRepository;
            _context = context;
        }

        private async Task<List<LoginHistoryDto>> _QueryData()
        {
            var _Que = _MainRepo.GetList();

            var invoices = await Map(_Que).ToListAsync();
            return invoices;
        }

        [HttpGet("[action]")]

        public async Task<IActionResult> GetAll()
        {
            var invoices = await _QueryData();
            return Ok(invoices);
        }

        // Mapping the MainInvoice to MainInvoiceDto

        private IQueryable<LoginHistoryDto> Map(IQueryable<LoginHistory> _Que)
    }
}

```

```

using Microsoft.AspNetCore.Mvc;
using FileUploadAPI.Repositories;
using Microsoft.EntityFrameworkCore;
using FileUploadAPI.Data;
using FileUploadAPI.Dto;

namespace FileUploadAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UserManagementApiController : ControllerBase
    {
        private readonly IUserManagementRoleRepository _MainRepo;
        private readonly ILogger<LoginHistoryController> _logger;
        private readonly AppDbContext _context;

        0 references
        public UserManagementApiController(IUserManagementRoleRepository mainInvoiceRepository, AppDbContext context, ILogger<UserManagementApiController> logger)
        {
            _MainRepo = mainInvoiceRepository;
            _context = context;
            _logger = logger;
        }

        0 references
        private async Task<List<LoginHistoryDto>> „QueryData()
        {
            var _Que = _MainRepo.GetList();

            var invoices = await Map(_Que).ToListAsync();
            return invoices;
        }
    }
}

```

```

using FileUploadAPI.Dto;
using FileUploadAPI.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Configuration;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Threading.Tasks;
using FileUploadAPI.Utilities;

namespace FileUploadAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    3 references
    public class RegisterController : _AbsAuthController<RegisterController>
    {
        private readonly UserManager<AppUser> _userManager;
        private readonly RoleManager<AppRoles> _roleManager;
        private readonly IConfiguration _configuration;

        // Define valid roles
        private static readonly UserRole[] ValidRoles =
        {
            UserRole.User,
            UserRole.Admin,
            UserRole.Verifier,
            UserRole.SuperAdmin
        };

        0 references
        public RegisterController(
            UserManager<AppUser> userManager,
            RoleManager<AppRoles> roleManager,
            ILogger<RegisterController> logger,
            IConfiguration configuration)
            : base(logger)
        {
            _userManager = userManager;
            _roleManager = roleManager;
            _configuration = configuration;
        }
    }
}

```

Assembly RepoBaseModelCore, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null

```
using System;
using System.Linq;
using System.Linq.Expressions;
using System.Threading.Tasks;

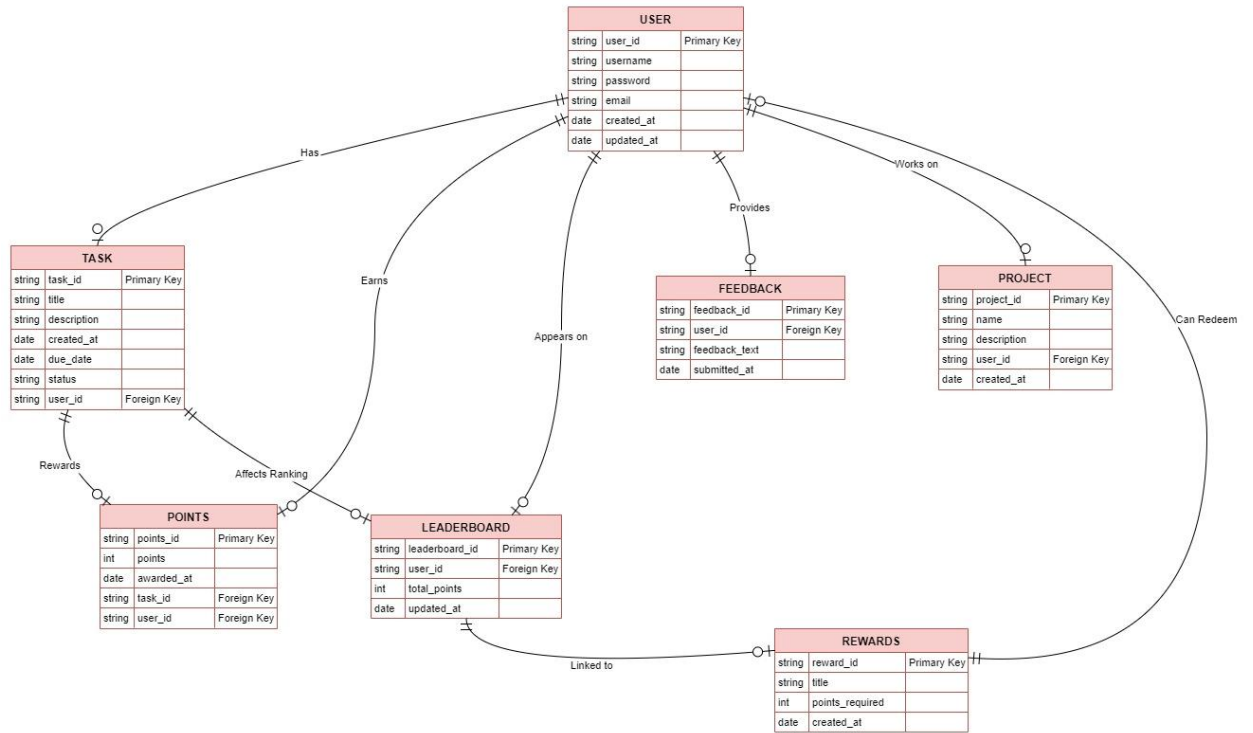
namespace RepoBaseModelCore;

public interface IGeneralRepository<Entity, PKType> : IDisposable where Entity : class
{
    bool Include(string RelTables);
    bool Include(string[] RelTables);
    IQueryable<Entity> FilterDeleted();
    IQueryable<Entity> FilterActive();
    IQueryable<Entity> GetList(bool IncludeDeleted = false);
    IQueryable<Entity> GetListWithInactive(bool IncludeDeleted = false);
    IQueryable<Entity> FindBy(Expression<Func<Entity, bool>> predicate);
    Task<Entity> FindAsync(PKType id);
    Entity FirstOrDefault();
    Entity GetDetail(PKType id);
    bool AddDefaultsInsert(Entity Data);
    bool Insert(Entity Data);
    bool AddDefaultsUpdate(Entity Data);
    bool Update(Entity Data);
    bool Save();
    bool Delete(Entity Data);
    Task<Entity> FirstOrDefaultAsync();
    Task<Entity> GetDetailAsync(PKType id);
}
```

No issues found

3.2.3. Database

3.2.3.1. Entity Relationship Diagram:



SQLQuery1.sql - DESKTOP-4JQ1TPB\SonicPoints (DESKTOP-4JQ1TPB\LEGION (59)) - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

Object Explorer

- DESKTOP-4JQ1TPB (SQL Server 16.0.1)
- Databases
 - System Databases
 - Database Snapshots
 - SMS
 - SonicPoints
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo_EFMigrationsHistory
 - dbo.Feedbacks
 - dbo.Leadboards
 - dbo.Points
 - dbo.Projects
 - dbo.ProjectTaskItems
 - dbo.ProjectTasks
 - dbo.Redems
 - dbo.Reports
 - dbo.UserProjects
 - dbo.Users
 - Dropped Ledger Tables
 - Views
 - External Resources
 - Synonyms
 - Programmability
 - Query Store
 - Service Broker
 - Storage
 - Security
 - Security
 - Server Objects
 - Replication
 - Always On High Availability
 - Management
 - Integration Services Catalogs

SQLQuery1.sql - D:\Q1TPB\LEGION (59)

```
--SELECT TOP (1000) [UserID]  
,[UserName]  
,[UserEmail]  
,[Password]  
,[UserRole]  
FROM [SonicPoints].[dbo].[Users]
```

100 %

Results Messages

	UserID	UserName	UserEmail	Password	UserRole
1	1	Sakchyan	sakchyan@gmail.com	123456	
2	2	void	void@gmail.com	\$2a\$11\$zHcaMVjQAWtWKAz2tc1FuF6QzhQ6MzviWwpAbbs...	
3	3	new	new@gmail.com	\$2a\$11\$odpQAz15OCgmBpaFaDSeByPqZs11O.71BIY5NN...	

Query executed successfully.

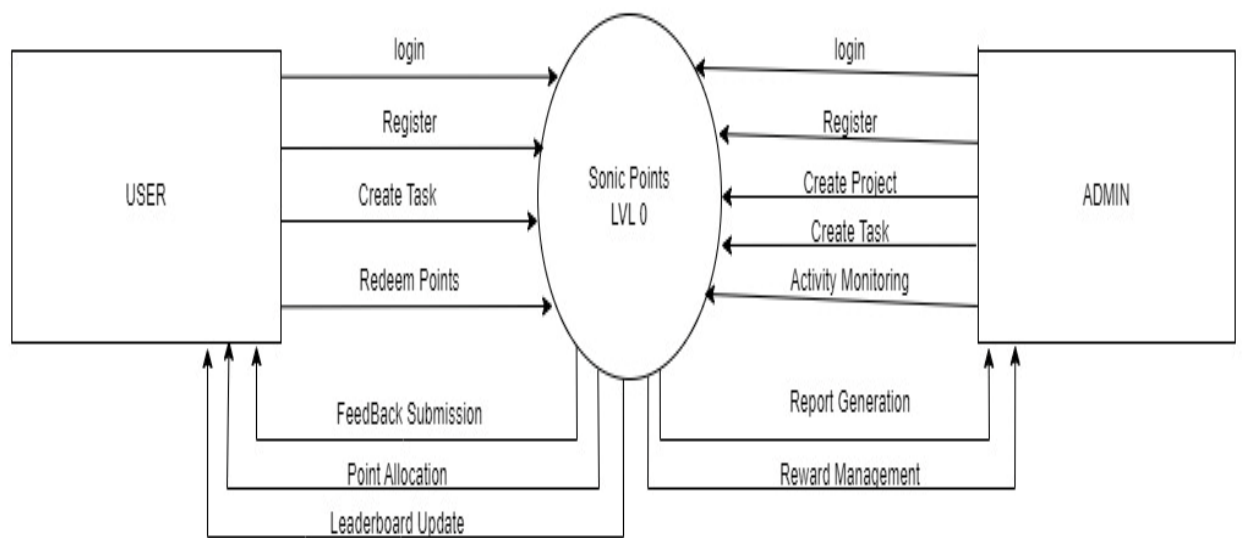
DESKTOP-4JQ1TPB (16.0 RTM) | DESKTOP-4JQ1TPB\LEGION... | SonicPoints | 00:00:00 | 3 rows

Ready | Ln: 1 | Col: 1 | Ch: 1 | INS

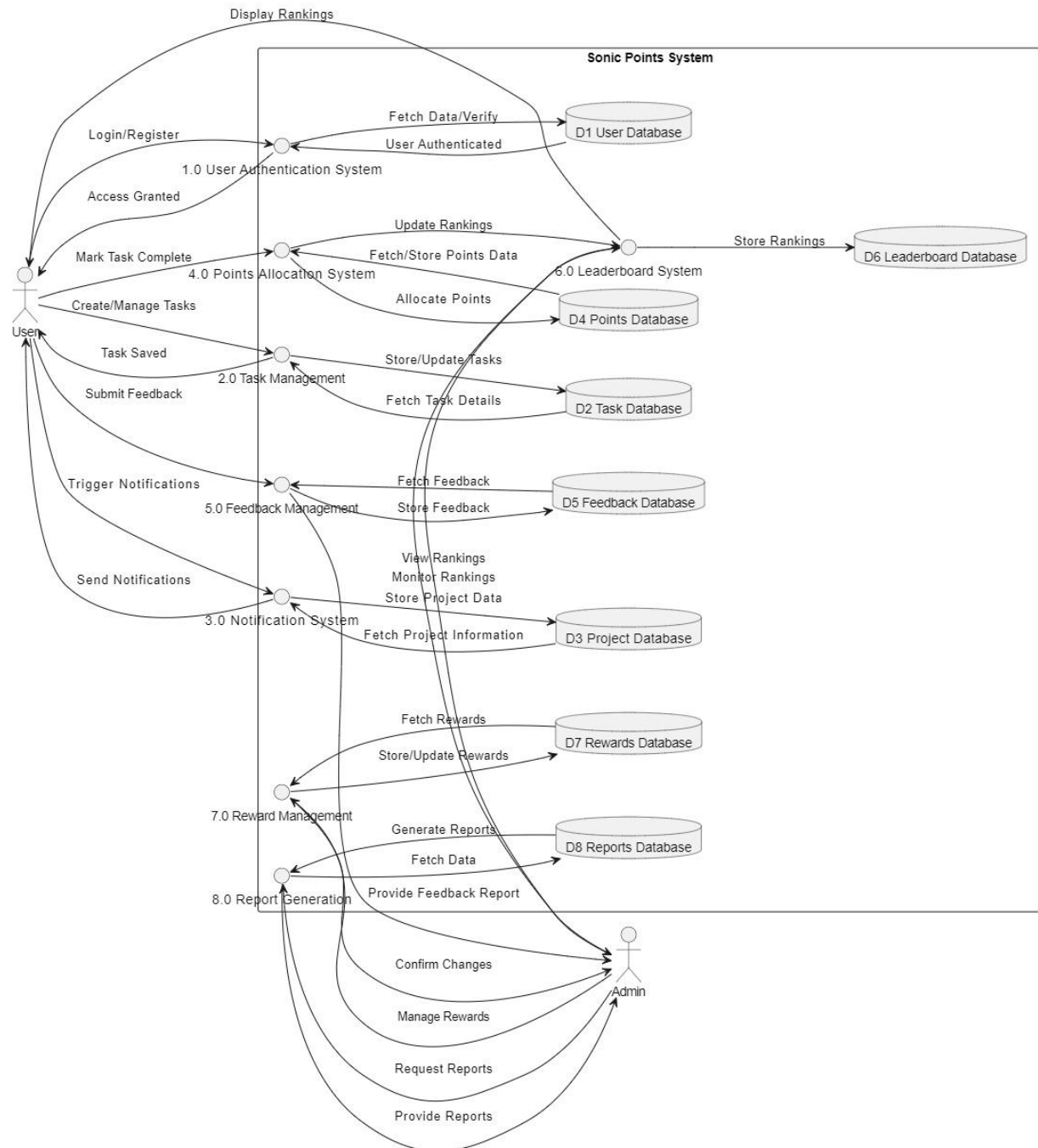
3.3. Development Till Date:

3.3.1. Data Flow Diagram:

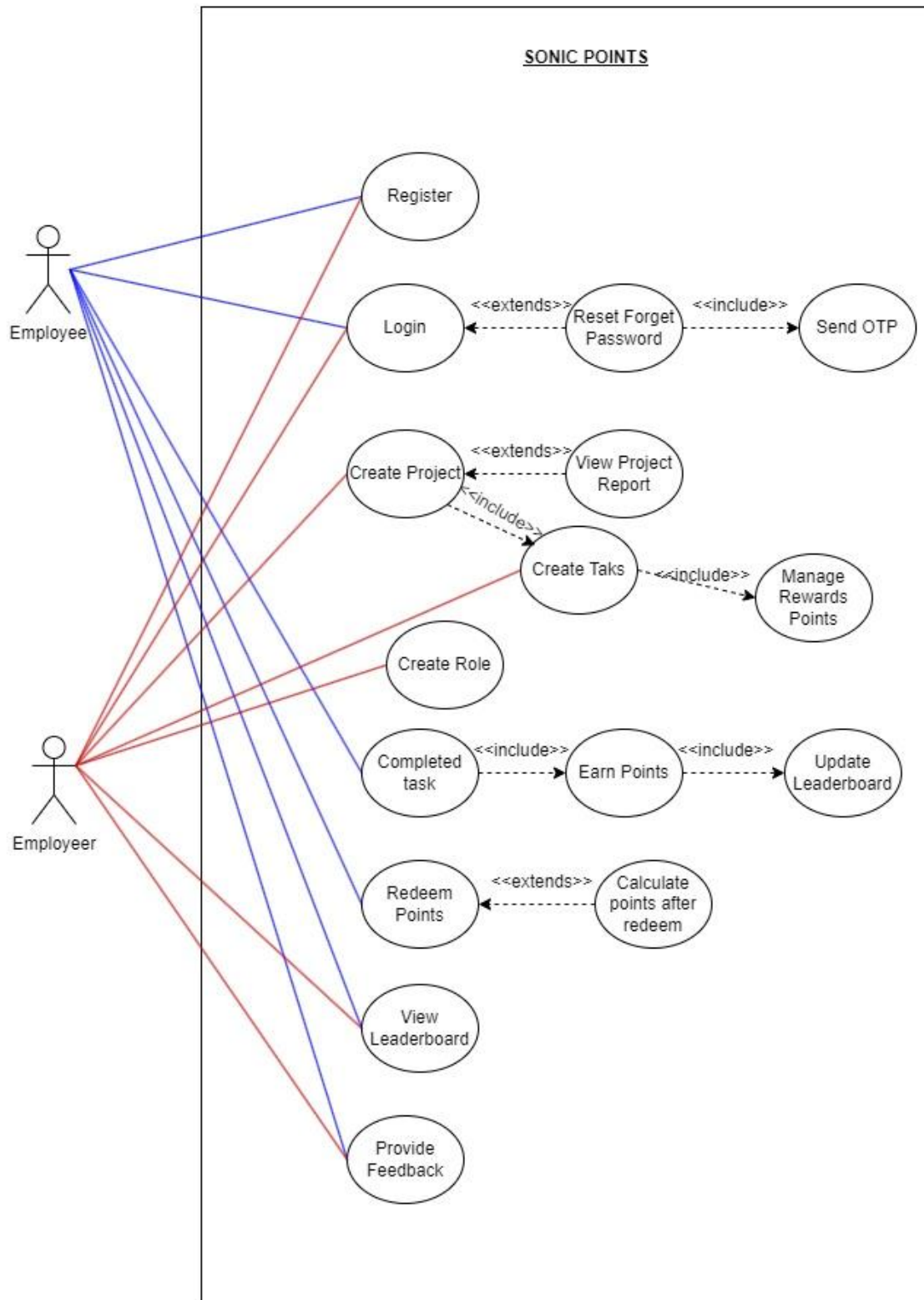
Data Flow Diagram Level 0:



Data Flow Diagram Level 1:



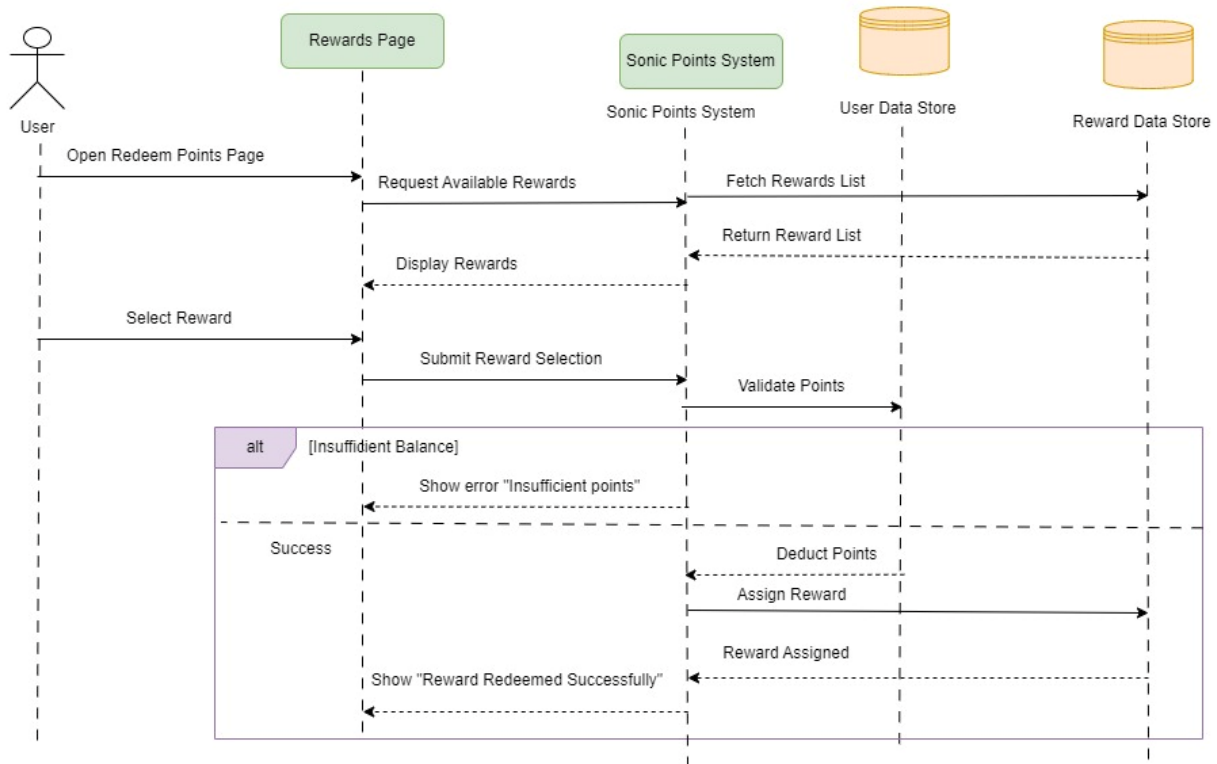
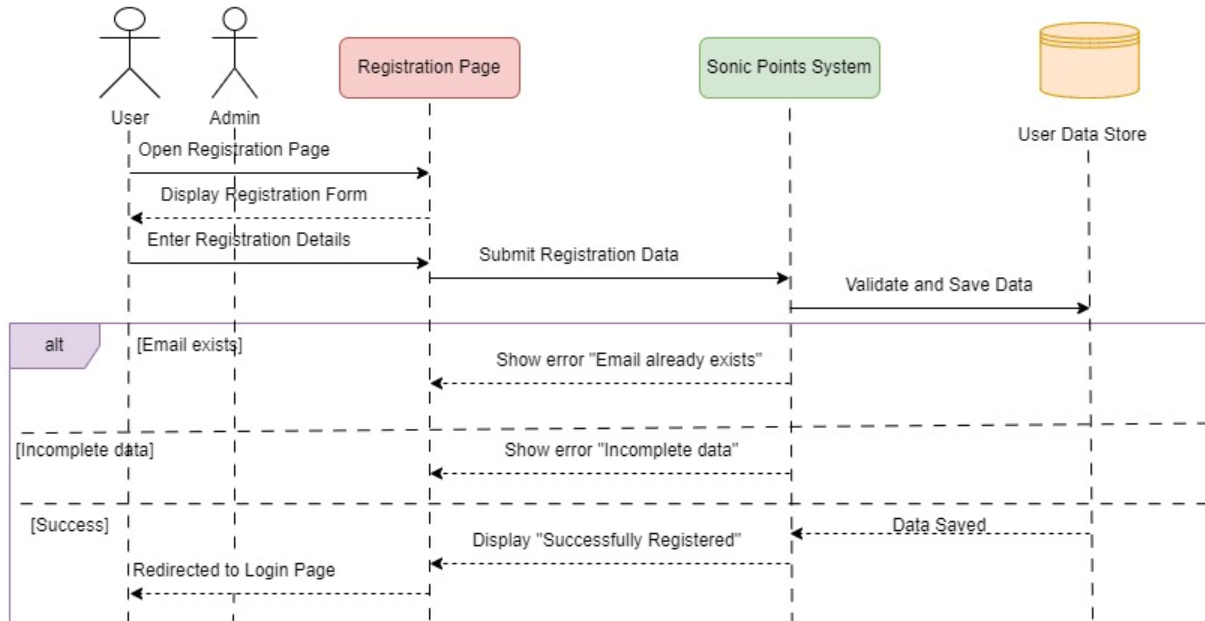
3.3.2. Use Case

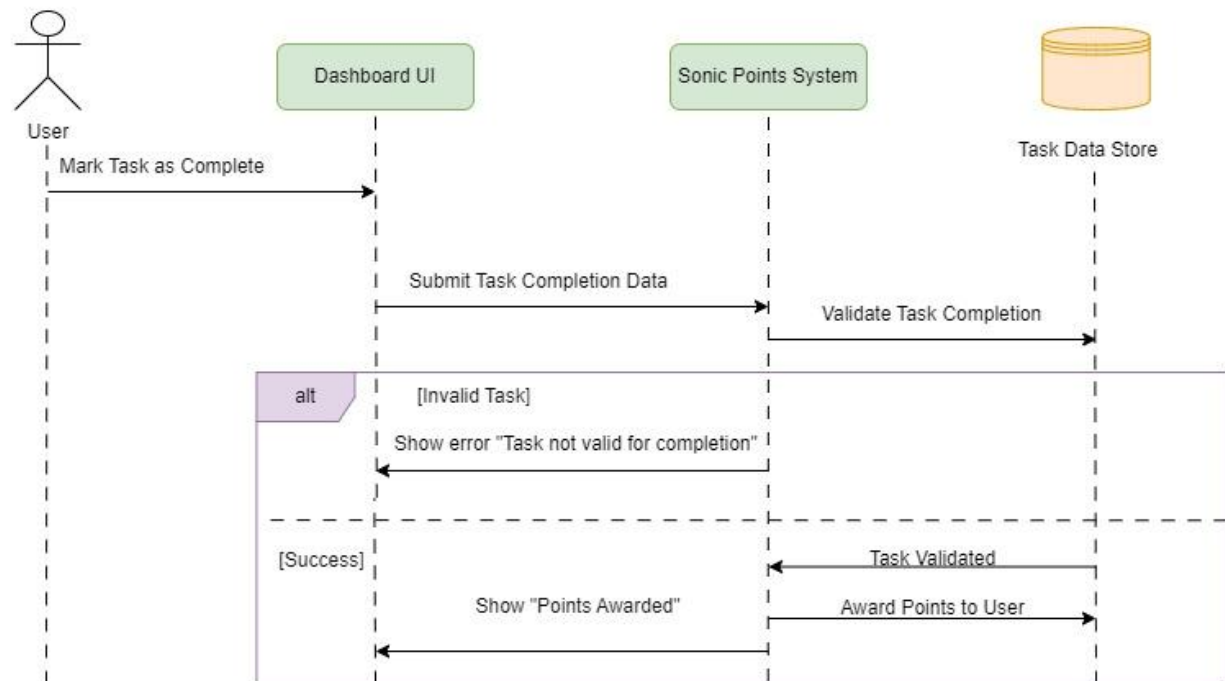
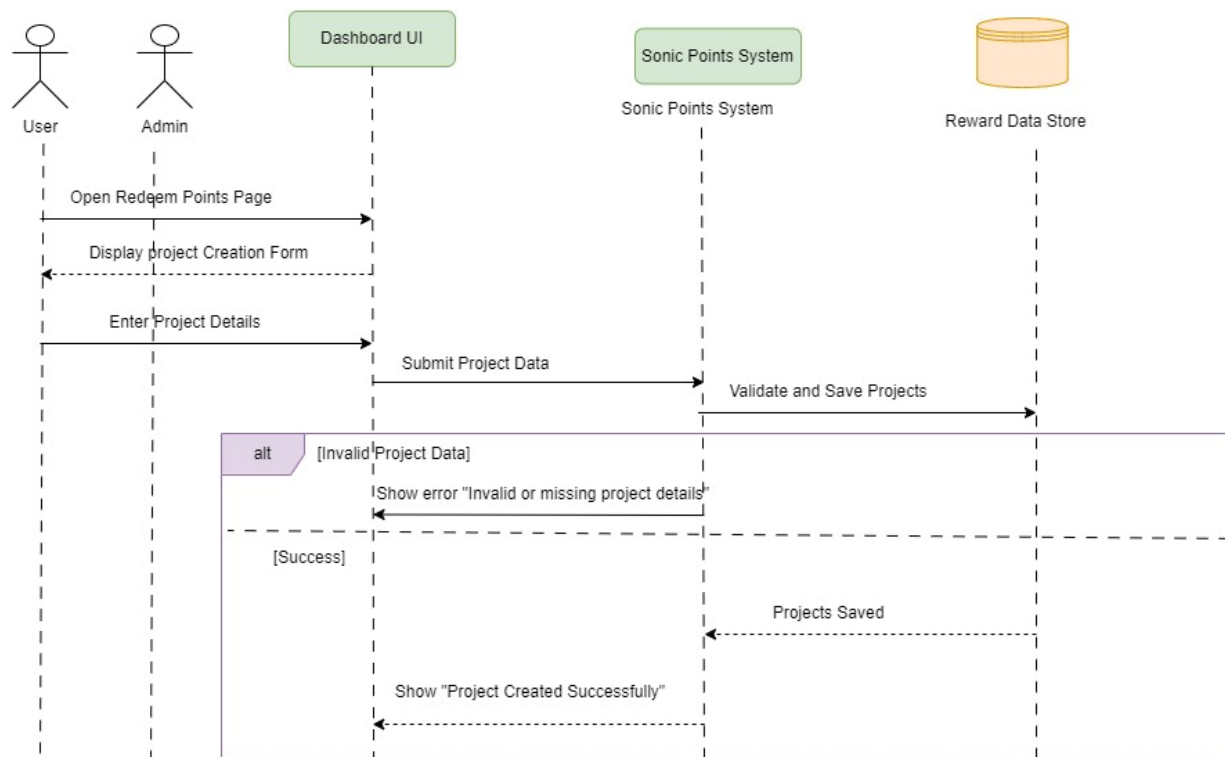


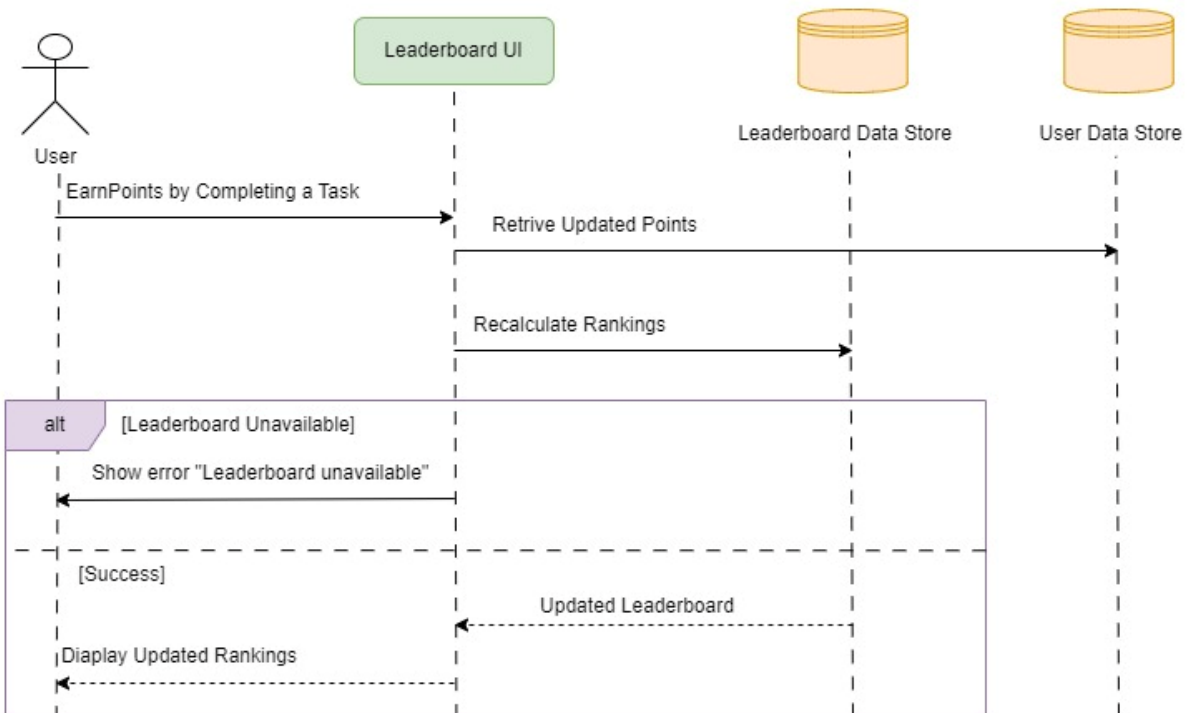
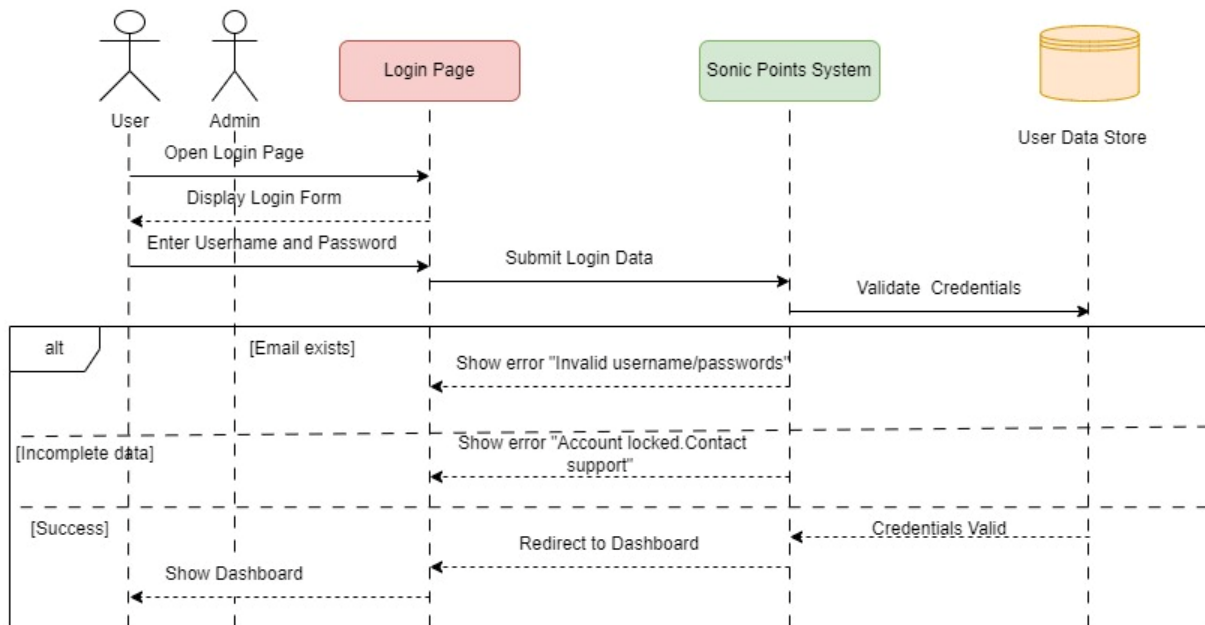
3.3.3. High Level Use Case & Extended Use Case:

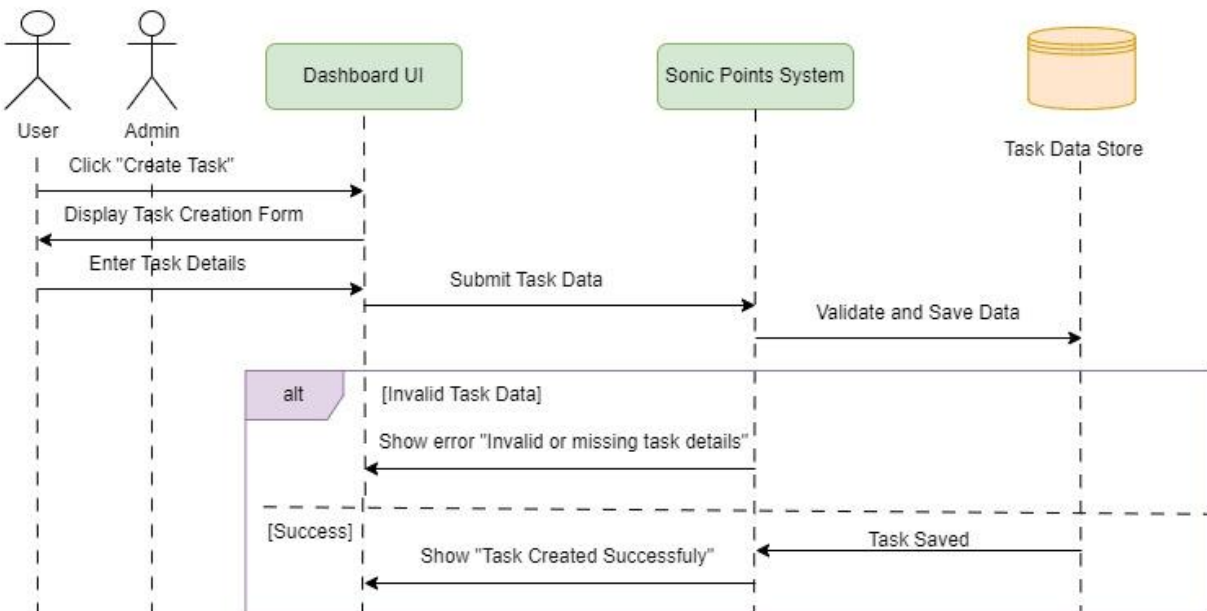
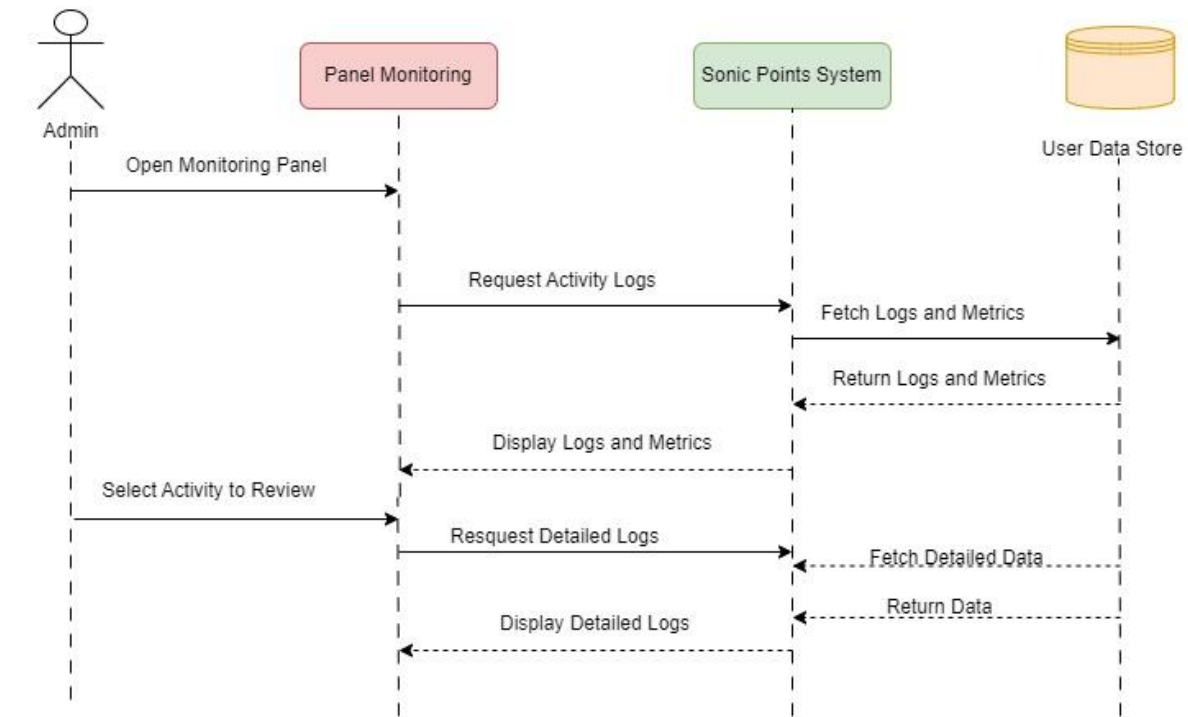
(VIEW)

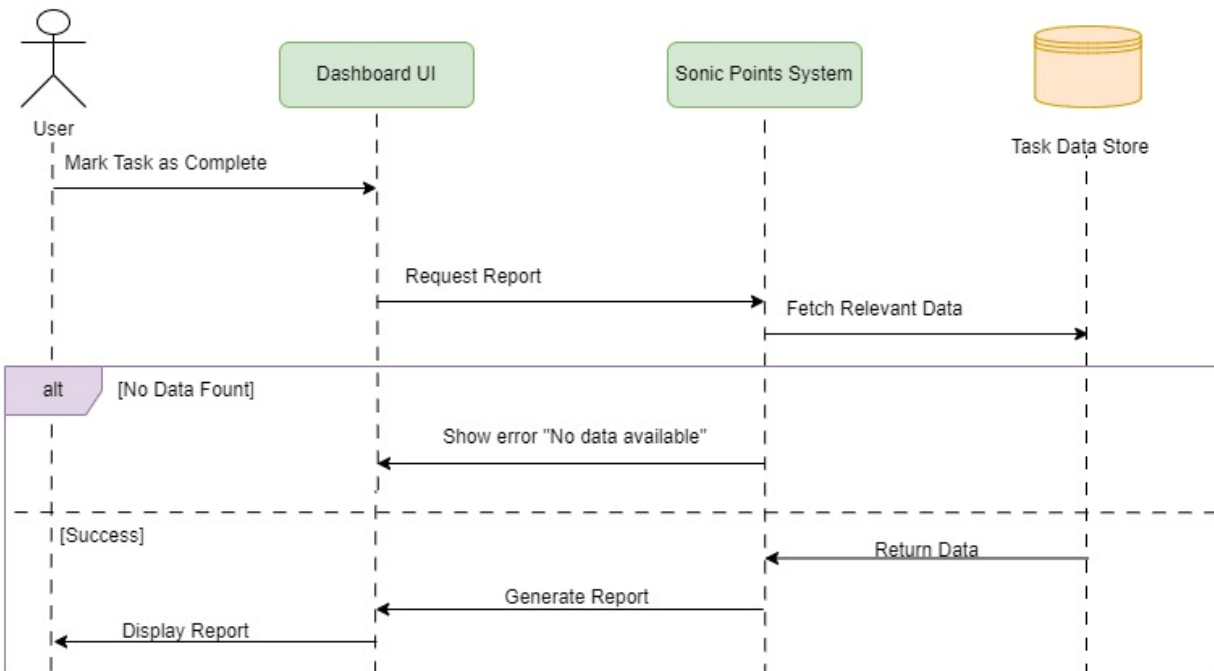
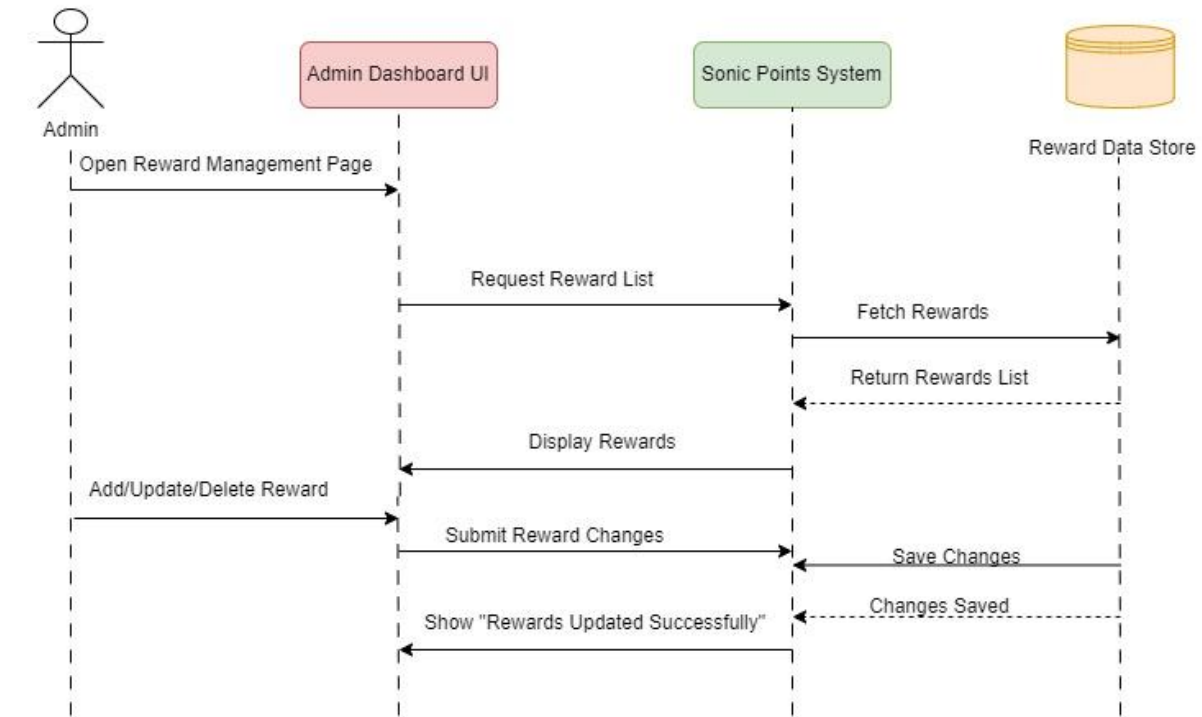
3.3.4. Sequence Diagram:



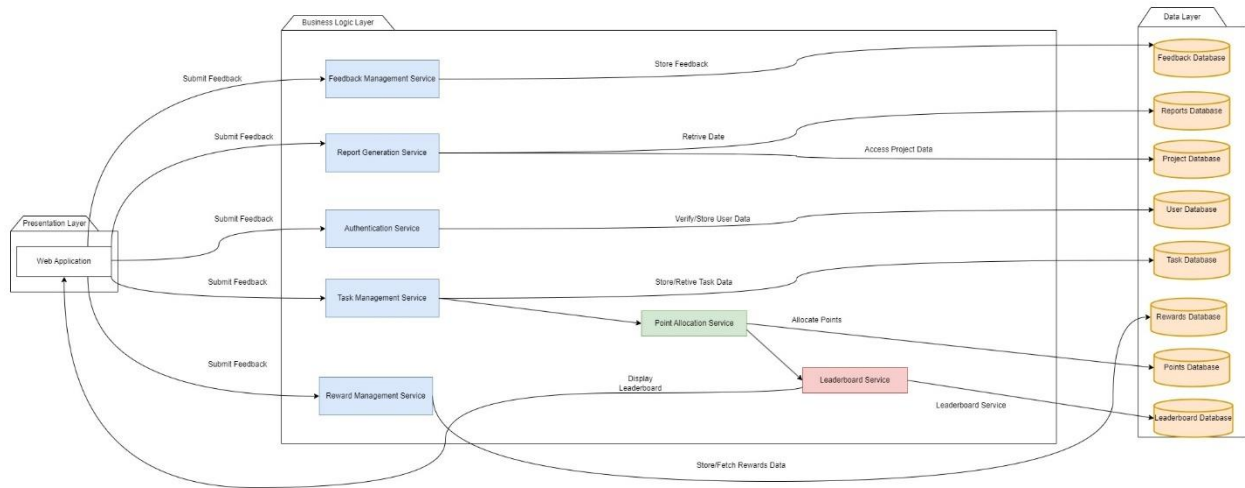




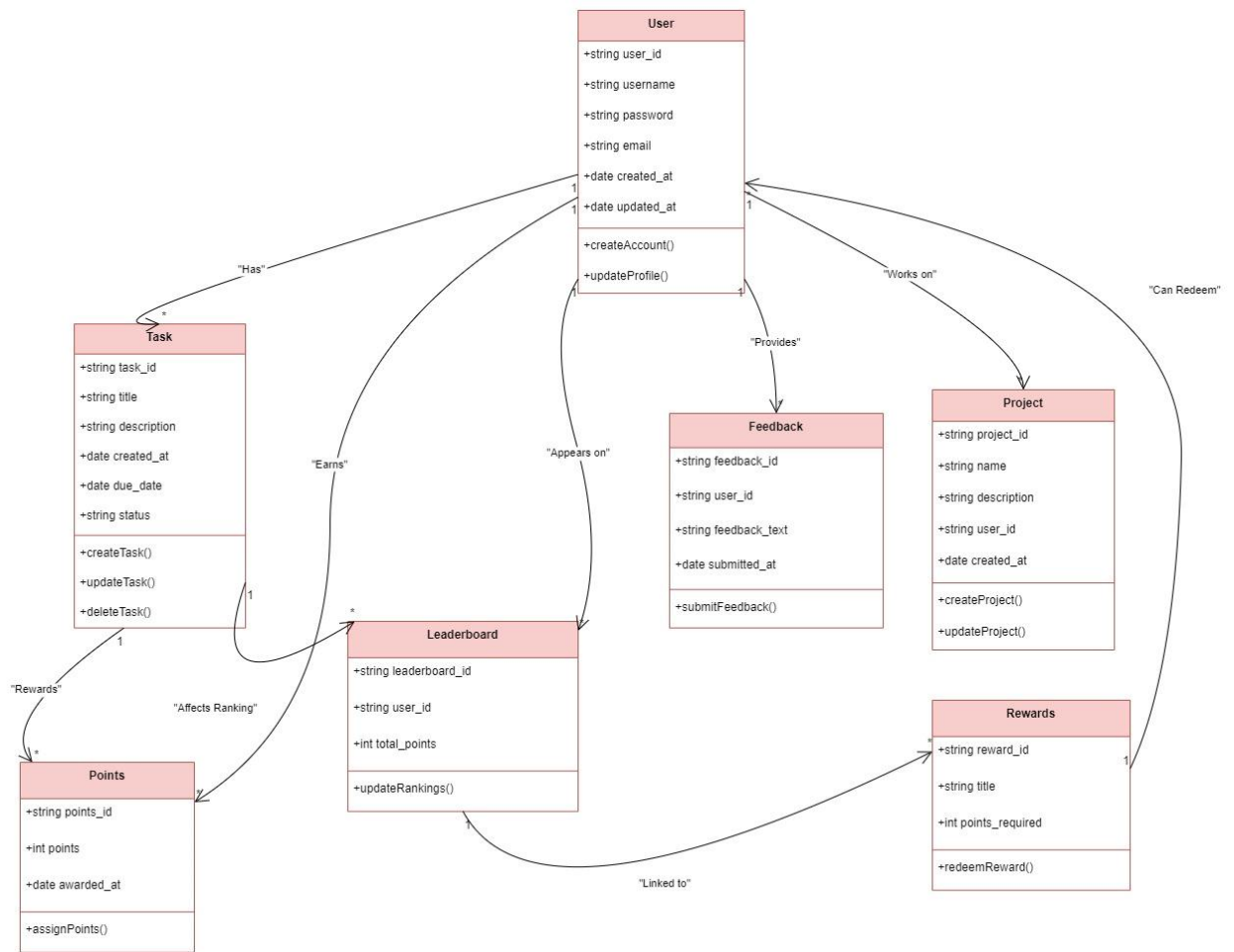




3.3.5. Architecture Diagram:



3.3.6. Class Diagram:



3.3.7. Normalization

Normalization is one of the key concepts behind database design, it makes the data less complex and the way to access the data simpler. The denormalization is a technique that tries to minimize the amount of duplicated data in a database by fusing two or more related tables together in a single table allowing better data persistence and quicker data retrieval.

3.3.8. Progress Table

	Tasks	Status	Progress
1	Research & Initiation	✓	100%
	Project Research	✓	100%
	Conduct Literature Review	✓	100%
	Gather Existing Research Data	✓	100%
	Identify Risks and Contingencies	✓	100%
	Requirements & Scope Finalized	✓	100%
	Finalize Proposal Report and Submit	✓	100%
2	UI Development	✓	100%
	Exploring UI Components	✓	100%
	Development of Frontend Components	✓	100%
	Integrating and Finalizing Frontend	✓	100%
3	Backend Development	Partial	60%
	Database Design & Development	✓	80%
	Development of API	✓	40%
	Authentication & Authorization	X	100%
	CRUD and Authentication Completed	✓	100%
	Interim Report Preparation	✓	100%
	Interim Report Completion	✓	100%
4	API & Web Integration	X	0%
	Setup of Templates & Dependencies	X	0%
	Web API Development	X	0%
	Final Sync Evaluation	X	0%
5	Final Adjustments	X	0%
	Testing & Debugging the Entire System	X	0%
	System Optimization	X	0%
	Completion of Sonic Points	X	0%
	Final Documentation Preparation	X	0%
	Final Report Submission	X	0%

4. Further Development:

1. Backend Development (Remaining 40%)

Though a lot has been done in backend development, the following are subtasks that need focused attention to be completed:

- Authentication & Authorization (40% Complete):
 - Finish user role-based access control (RBAC).
 - Enhance security around sensitive user data.
 - Test and debug to ensure secure login/logout processes.
- API Development (60% Remaining):
 - Finish implementing the remaining API endpoints.
 - Ensure APIs are secure, scalable and optimized for performance.
 - Test API integrations with dummy data to ensure functionality.

2. API & Web Integration (0%)

The integration phase is an important step in creating a proper bridge between the backend services and the frontend user interface. Tasks involve:

- Setup of Templates & Dependencies:
 - Install and configure all the dependencies required by the integration of APIs and web application.
 - Set up project scaffolding to streamline the integration process.
- Web API Development:
 - Design and develop endpoints for data exchange between the client and server.
 - Ensure compatibility with the frontend interface and establish efficient data flow mechanisms.

- Final Sync Evaluation:

- Perform synchronization testing to verify that the frontend and backend are fully integrated.
- Resolve inconsistencies in data flow or user interactions.

3. Final Adjustments (0%)

This stage deals with the refinement and optimization of the project before the final delivery. The main activities here are:

- Testing & Debugging of Complete System:

- Unit test, integration test, and system test to find and fix all bugs.
- Stress and performance tests to ensure the stability of the system under load.

- System Optimization:

- Improve API and database query performance.
- Improve responsiveness of the UI and reduce page load times.

Sonic Point Completion

- Complete any outstanding or high-priority enhancements raised in previous cycles.

Final Documentation Preparation

- Collate all technical documentation including architecture, development process, and user guides
- Ensure that it is clear and comprehensive for end-users and future developers.

Final Report Submission

- Prepare the final report summarizing the project objectives, development progress, challenges faced, and the outcomes.
- Send the report to relevant stakeholders for review.

Timeline for Completion

Based on the progress and tasks involved, the following timeline is suggested:

- Backend Development: 2 weeks
- API & Web Integration: 3 weeks

- Final Adjustments:2 weeks

With dedicated effort and efficient collaboration, the project can be delivered within the next 7 weeks.

5. References

(Zichermann G., 2011)

(B., 2012)

(Kaplan R.S., 1996)

Harlow, J. (2024). *What is a Key Performance Indicator (KPI)?* - KPI.org. [online] KPI.org. Available at: <https://www.kpi.org/kpi-basics/> [Accessed 4 Dec. 2024].

What is a Key Performance Indicator (KPI)? Guide & Examples. [online] Available at: <https://www.qlik.com/us/kpi> [Accessed 4 Dec. 2024].

Psu.edu. (2015). *KPI and Metrics | Office of Planning, Assessment, and Institutional Research.* [online] Available at: <https://opair.psu.edu/planning-handbook/kpi-metrics/> [Accessed 4 Dec. 2024].

Kinsey, S. (2018). *What is a Key Performance Indicator (KPI).* [online] SimpleKPI.com. Available at: <https://www.simplekpi.com/Resources/Key-Performance-Indicators> [Accessed 4 Dec. 2024].

<https://oauth.net/2/>

<https://jwt.io/>

<https://www.smashingmagazine.com/2018/01/understanding-using-rest-api/>

<https://restfulapi.net/>

<https://www.okta.com/blog/2020/03/best-practices-for-securing-rest-apis/>

<https://www.lucidchart.com/pages/database-design>

<https://axios-http.com/>

<https://www.postman.com/>

Jira (<https://www.atlassian.com/software/jira>)

Sentry (<https://sentry.io/>)

6. Appendix

6.1. Hardware Requirements

1. Development Machine:

- Processor: Intel i5 or equivalent
- RAM: At least 8 GB (16 GB is recommended for increased performance)
- Storage: SSD storage with a minimum of 256 gigabytes of free space.

2. Testing Devices:

- Testing Devices: Testing will be performed on multiple browsers (Chrome, Firefox, Edge) and operating systems (Windows, macOS), ensuring that the web API functions properly in various environments. Functionalities can be verified using emulators(PRD) or API testing tools(Postman).

6.2. Software Requirements

Backend Development:

Technology	Details
Framework	.NET (preferably .NET Core or ASP.NET) v8.0
Database Management System	SQL Server management studio v.20
Development Tools	Visual Studio v 8.0
Frontend Development	HTML/CSS, JavaScript, Bootstra

Tool	Git (with platforms like GitHub or GitLab for repository hosting)
API Testing	Postman for testing backend APIs
Project Management and Collaboration	Trello for task management
Design Tools	UI/UX Design: Figma for creating wireframes and mockups

Communications Interfaces:

Protocols:

HTTP/HTTPS: Protocols for UI and middleware communications.

SendGrid: For email deliveries, such as secret PIN notifications..

Data Formats:

Microsoft Sql Database: It is for storing all details in the systems.

Swagger API: For organized communication between the frontend and the backend.

Security:

✓ Passwords are hashed securely.

6.3. Key Dissimilarities of Sonic Points:

- **KPI-Driven Approach:** Unlike other platforms like Trello, Todoist, Asana, and Habitica, which do not offer KPI tracking, Sonic Points enables users to track performance through KPIs. This feature allows users to measure their progress and optimize workflows based on real-time data, something most other platforms do not emphasize.
- **Gamification & Rewards:** While Habitica does incorporate gamification, it focuses on turning tasks into quests in an RPG format. Sonic Points, however, uses a points system, leaderboards, and rewards tied directly to task completion, creating a more structured and competitive environment. Platforms like Trello and Todoist completely lack gamification, focusing only on task organization.
- **Individual vs. Team Focus:** Sonic Points is tailored for individual productivity and personal goal setting, whereas Trello and Asana are designed for team collaboration. Sonic Points offers personalized performance insights and helps individuals optimize their own workflows, which is a feature lacking in team-centered tools like Trello and Asana.
- **Competition and Engagement:** Sonic Points introduces a live leaderboard where users can compete based on their task completion, fostering a sense of healthy competition. In contrast, Trello, Todoist, and Asana focus on task management without any competitive elements.
- **Reward System:** The reward system in Sonic Points is centered on task completion and redeemable points, while platforms like Todoist and Asana do not offer any points or rewards for completed tasks. Habitica, while gamified, rewards users with in-game items rather than offering meaningful real-world rewards tied to task completion.
- **Data-Driven Decisions:** Sonic Points stands out by offering detailed, real-time performance analytics and predictive trends to help users improve task efficiency. Other tools like Trello and Todoist lack this level of performance tracking and predictive insights, which are vital for users focused on improving their personal productivity.
- **Customization:** Sonic Points offers a high level of customization with its KPI tracking and task management. Other tools are limited in this respect. Asana and Trello offer customization in organizing tasks, but they do not allow for tracking personal KPIs or tailoring metrics to the user's specific goals.

6.4. REQUIREMENTS Gathering:

❖ Planning

❖ Goal: Establish the project Scope, Timeline, and the Development Resources.

❖ Activities:

- o Develop project milestones and deliverables.
- o Use the Kanban board to organize tasks according to priority and add due dates.
- o Make sure that the Kanban board represents the flow of tasks (e.g., "Backlog," "In Progress," "Completed").

❖ Design

o Objective: Build Application Architecture and User Interface

❖ Activities:

- o Application architecture (o backend (e.g.,. NET) and frontend via React.
- o Design wireframes and mockups of the user interface, focusing on user experience (UX) and accessibility.
- o Define the database schema to store tasks, user information, and related data.

❖ Development

❖ Approach: Develop the application incrementally with working features built in smaller increments.

❖ Activities:

- o Define development environment and version control (like Git)
- o Develop core functionality from user stories, including creating tasks, assigning tasks, and task progress.
- o This Kanban board will help track each task and making sure that there

are no bottleneck while work is performed.

❖ Testing

❖ Test whether the application works properly without errors.

❖ Activities:

- o Unit tests for individual components and features.
- o Perform integration testing to ensure that various components of the application interact properly.
- o Conduct User Acceptance Testing (UAT) with end-users to receive feedback and highlight any problems.
- o Keep the progress of test cases and bug fixes visually on the Kanban board for no missed tasks.

❖ Deployment

❖ Goal: Deploy the app in a production environment for users.

❖ Activities:

- o Set up the server environment for the application (cloud hosting, web server setup).
- o Deploy the application and do final verification to ensure everything working fine.
- o User training and documentation support to drive adoption

❖ Maintenance and Support

❖ Purpose: Repeatedly support and update so that the application still works.

❖ Activities:

- o User feedback: Observe the application for bugs, performance issues, etc.
- o Conduct periodic updates for bug fixes, feature improvement and new

functionality based on user feedback.

- o Manage maintenance work via the Kanban board, prioritizing issues and tracking status.

❖ Evaluation and Feedback

- ❖ Objective: Evaluation of project against goals and gaining user feedback for improvement.

❖ Activities:

- o Survey or conduct interviews with users post-launch to assess their experience.

- o Review usage data to understand where you can improve or add functionality

- o Assess and consider feedback through the Kanban board for process refinement. Requirements for Software and Hardware

6.5. Non Functional Requirements

1. Performance and Scalability

- **Description:** The system should be able to handle a large number of users and tasks efficiently without degradation in performance.
- **Requirements:**
 - The system should handle up to **10,000 concurrent users** without significant performance issues.
 - Pages should load within **2-3 seconds** under normal load.
 - The task processing system should scale horizontally to support future growth.

2. Security

- **Description:** The system must ensure that users' data and interactions are secure.
- **Requirements:**
 - Implement **SSL encryption** for all communications between the client and server to prevent data interception.
 - User passwords must be **hashed and salted** before storage (e.g., using **bcrypt**).
 - All user data must be stored in a secure database with access controls and periodic audits.
 - The system must adhere to **GDPR** and **Data Protection** regulations if applicable.
 - Implement **two-factor authentication (2FA)** as an option for added security.

3. Availability and Reliability

- **Description:** The system should be available and reliable at all times to ensure a seamless user experience.
- **Requirements:**
 - The system must have an **uptime of 99.9%** or better.
 - Implement **auto-scaling** and load balancing to ensure high availability during peak usage times.
 - Provide **backups** of user data at regular intervals to prevent data loss.
 - The system should be able to **recover** within **30 minutes** in case of a failure.

4. Usability

- **Description:** The system should be intuitive, easy to use, and provide a great user experience.
- **Requirements:**
 - The user interface should be simple, clean, and easy to navigate, with no more than **3 clicks** to perform any primary action.
 - The system should provide clear, helpful error messages and tooltips to guide the user through tasks.
 - Implement accessibility features such as **keyboard navigation** and **screen reader support** to accommodate all users, including those with disabilities.
 - The system should be **mobile-friendly** and responsive, providing an optimal user experience across different devices.

5. Maintainability

- **Description:** The system should be designed for ease of maintenance, updates, and troubleshooting.
- **Requirements:**
 - The codebase should follow standard best practices, using clean and well-documented code.
 - The system should have comprehensive **unit tests** and **integration tests** to ensure smooth updates.
 - Errors and logs should be collected and stored in a centralized logging system for efficient troubleshooting.
 - The system should allow for easy updates to the user interface, tasks, and leaderboard without major disruptions.

6. Legal and Regulatory Compliance

- **Description:** The system should comply with applicable laws, standards, and regulations.
- **Requirements:**
 - The system should be compliant with data protection regulations such as **GDPR**, **CCPA**, and any other applicable laws in the jurisdictions where it operates.

- Ensure proper consent for data collection and clear privacy policies.
- The system must adhere to industry-standard security protocols for user data protection.

7. Documentation

- **Description:** Comprehensive documentation should be available for both end-users and developers.
- **Requirements:**
 - Provide **user guides** and **help documentation** within the application.
 - Maintain a detailed **API documentation** for integration with third-party systems.
 - Include clear **technical documentation** for developers to maintain and extend the system.

6.6. REQUIREMENTS SPECIFICATIONS OF AN SRS:

The Sonic Points System is a game-based project management system modeled on the inspiration of KPI-driven solutions and methodologies that achieve sea changes in levels of productivity and engagement. Conventional systems or typical ones lack motivation and engagement on the ground level performance, thus keeping big voids in employees' efficiencies and the achievements of results. This will fill the gap by integrating a novel platform of advanced tools and features that help in collaboration, healthy competition, and transparency in task management.

The Sonic Points System provides a more formalized mechanism of task delegation, tracking, and closure with a dynamic way of allocating points in addition to real-time leaderboards. Additionally, rewards and key performance indicators act as motivators towards achieving set goals; hence, a culture of continuous improvement is created. Management will be enlightened at both the individual and team levels in order to inform their decision-making processes.

Equipped with the core functionalities of point-based rewards, live ranking, feedback management, and report generation, the system tries to smoothen the processes in an organizational setting and increase engagement and accountability in the process. Blending productivity tools with gamification strategies, Sonic Points aims at developing a smarter, more interactive, and efficient work environment that advances the cause of individual and collective growth.

6.7. High Level Use Case

Use Case	Registration
Actor	User
Description	Enables new users to sign up and create an account. The user opens the registration form and provides valid input. Upon successful registration, a user account is created.

Use Case	Secure Login
Actor	User/Admin
Description	Users and admins log into the system securely. The user enters login credentials. Access is granted if the credentials are valid and exist in the system.

Use Case	Task Creation
Actor	User/Admin
Description	Users add tasks under specific projects. The user selects a project. Task is saved under the project if the user is logged in and the project exists.

Use Case	Points Allocation
Actor	User
Description	Users are awarded points for completing tasks. The user marks a task as complete. Once the task is marked complete, points are added to the user's account.

Use Case	Project Creation
Actor	User
Description	Users can create projects to organize tasks. The system allows users to input project details and validates them before saving the project. The project is created successfully if the user is logged in and the input data is valid.

Use Case	Leaderboard Updates
Actor	User
Description	The system updates the leaderboard with the latest user rankings. The leaderboard reflects updated rankings once a user completes a task or earns points.

Use Case	Redeem Points
Actor	User
Description	Users redeem accumulated points for available rewards. The user selects the "Redeem Points" option. If the user has sufficient points, points are deducted and rewards are assigned.

Use Case	Feedback Submission
Actor	User
Description	Users provide feedback about their experience with the application. The user clicks the "Feedback" option. Feedback is saved for review if the user is logged in.

Use Case	Reward Management
Actor	Admin

Description	Admins add, update, or delete rewards that users can redeem. The admin accesses the rewards section. Rewards are updated if admin privileges are verified.
-------------	--

Use Case	Report Generation
Actor	Admin
Description	System generate performance or activity reports. The admin requests a report. The system generates the report if admin privileges are verified and data is available.

Use Case	Activity Monitoring
Actor	Admin
Description	Admin tracks user activity for performance and issue resolution. The admin logs into the monitoring panel. The system identifies any issues or irregularities.

Use Case	Create Role
Actor	Admin
Description	The admin creates a new role to define user permissions and access levels. They navigate to the "Role Management" panel, input role details, and submit the form. The system validates, saves the role, and confirms successful creation.

6.8. Extended Use

Here's the full expanded use case list with alternative courses included for each use case:

1. Register

Use Case Name: Register

Actors: User, Admin

Description:

Employees and managers register by entering their personal details. This registration allows them to access their respective personalized dashboards.

Typical Course of Events:

Actor Action	System Response
1. Employee/manager fills the registration form.	2. System validates the entered data.
3. User submits the registration form.	4. System creates an account and saves the user details in the database.
	5. Displays a "Successfully Registered" message.
	6. User proceeds to log in.

Alternative Courses:

Condition	Response
Line 1: User enters incomplete data	Use Case Ends: System shows an error message and asks the user to re-enter and re-submit.
Line 2: Email already exists in the system	Use Case Ends: System displays an error message asking the user to provide a different email.

2. Create Role

Use Case Name: Create Role

Actors: Admin

Description:

The admin creates a new role to define specific permissions and access levels for system users. This ensures that different users have appropriate access and responsibilities within the system.

Typical Course of Events:

1. Admin navigates to the "Role Management" panel.
2. System displays the "Create Role" form.
3. Admin enters the role name, description, and selects permissions.
4. System validates the entered data.
5. Admin submits the form.
6. System saves the role to the database.
7. System displays a confirmation message: "Role created successfully."
8. Admin can proceed to assign the role to users.

2. Login

Use Case Name: Login

Actors: User, Admin

Description:

Users and admins log in by entering credentials. The system authenticates them to grant dashboard access.

Typical Course of Events:

1. User/Admin enters username and password.
2. System validates the credentials.
3. User submits the login form.
4. System authenticates and redirects the user/admin to their dashboard.

Alternative Courses:

Condition

Line 1: Invalid username/password

:Use Case Ends: System shows an error message and prompts the user to try again.

Line 2: Account locked due to multiple failed attempts

: Use Case Ends: System displays an account-lock message and suggests contacting support.

3. Task Creation

Use Case Name: Task Creation

Actors: User,Admin

Description:

Users create tasks under specific projects to organize their workload.

Typical Course of Events:

1. User selects a project and clicks "Create Task."
2. System displays a task creation form.
3. User fills out the task details.
4. System validates and saves the task under the project.

Alternative Courses:

Line 1: Invalid or missing task details

:Use Case Ends: System shows an error message asking the user to correct the input.

4. Points Allocation

Use Case Name: Points Allocation

Actors: User

Description:

Points are awarded to users when tasks are successfully completed.

Typical Course of Events:

1. User marks a task as complete.
2. System validates the task completion.
3. Points are awarded, and the user is notified.

Alternative Courses:

Line 1: Task not valid for completion

:Use Case Ends: System does not allocate points and notifies the user of the issue.

5. Leaderboard Updates

Use Case Name: Leaderboard Updates

Actors: User

Description:

System updates the leaderboard rankings when users earn points.

Typical Course of Events:

1. User earns points by completing a task.
2. System recalculates rankings.
3. The updated leaderboard is displayed to all users.

Alternative Courses:

Line 1: Leaderboard is inaccessible Use Case Ends: System notifies users about the unavailability and retries updating later.

6. Redeem Points

Use Case Name: Redeem Points

Actors: User

Description:

Users redeem points for rewards available in the system.

Typical Course of Events:

1. User selects "Redeem Points" option.
2. System displays available rewards.
3. User chooses a reward and confirms.
4. Points are deducted, and the reward is assigned to the user.

Alternative Courses:

Line 1: Insufficient points

:Use Case Ends: System shows an error message and prevents the redemption.

7. Feedback Submission

Use Case Name: Feedback Submission

Actors: User

Description:

Users provide feedback on their experience with the system.

Typical Course of Events:

1. User clicks "Submit Feedback."
2. System displays a feedback form.
3. User fills out and submits the form.
4. System saves the feedback and shows a success message.

Alternative Courses:

Line 1: Feedback form left incomplete
requests completion before submission.

Use Case Ends: System notifies the user and

8. Reward Management

Use Case Name: Reward Management

Actors: Admin

Description:

Admins manage rewards available for users to redeem.

Typical Course of Events:

1. Admin accesses the rewards management page.
2. System displays a list of rewards.
3. Admin adds/updates/deletes a reward.
4. System saves the changes and confirms.

9. Report Generation

9. Use Case Name: Report Generation

Actors: Admin

Description:

Admins generate reports on user activities or performance.

Typical Course of Events:

1. Admin selects "Generate Report" option.
2. System displays report types.
3. Admin chooses a type and confirms.
4. System generates the report and provides it for viewing/download.

Alternative Courses:

Line 1: No data available for the report

:Use Case Ends: System shows an error message and suggests alternative report types.

10. Activity Monitoring

Use Case Name: Activity Monitoring

Actors: Admin

Description:

Admins monitor user activities to ensure performance and resolve issues.

Typical Course of Events:

1. Admin logs into the activity monitoring panel.
2. System displays user activity logs and tools.
3. Admin selects a user or activity to review.
4. System shows detailed logs and metrics.

11. Project Creation

Use Case Name : Project Creation

Actor : User

Description:

Users create projects by entering relevant details such as project name, description, and deadline. The system validates the details and saves the project in the database. If the user is not logged in or inputs invalid data, the system displays an error message.

Typical Course of Events:

1. User clicks the "Create Project" button.
2. System displays the project creation form.
3. User fills in project details.
4. System validates the input data.
5. User submits the form.
6. If data is valid, the system saves the project in the Project Data Store.
7. System shows a success message: "Project Created Successfully."

6.9. Normalization

Unnormalized Form (UNF) and Converting to 1NF:

UNF: It is a type of database design in which data is stored directly in a single, unstructured table with repeating groups or a second or third value in a single column. Such structure creates duplication and inconsistency.

UserID, UserName, UserEmail, UserContact

{FeedbackID, FeedbackText, FeedbackSubmittedAt}

{ProjectID, ProjectName, ProjectStatus, ProjectStartDate, ProjectDueDate}

{TaskID, TaskName, TaskStatus, TaskStartDate, TaskDueDate}

Steps to Convert UNF to 1NF:

Remove Repeating Groups:

Transposes multiple values of a column (lists) into separate rows Each value should be a row in itself.

Ensure Atomicity:

Atomic (single-valued) nature: each cell should hold a single value. Multi-valued attributes are not permitted..

Ensure Uniqueness:

No two rows can be the same, and you will typically have a primary key (i.e. student id + course).

6.9.1. Identify Repeating Groups

Users provide Feedback and manage Projects. Columns: UserID, UserName, UserEmail, UserContact, {FeedbackID, FeedbackText}, {ProjectID, ProjectName, ProjectStatus, ProjectStartDate, ProjectDueDate}

Projects have multiple Tasks. Columns: ProjectID, {TaskID, TaskName, TaskStatus, TaskStartDate, TaskDueDate}

Removing Repeating Groups

- User-1 Table:

UserID, UserName, UserEmail, UserContact

- Feedback-1 Table:
- FeedbackID, FeedbackText, UserID*
- Project-1 Table:

ProjectID, ProjectName, ProjectStatus, ProjectStartDate, ProjectDueDate, UserID*

- Task-1 Table:

TaskID, TaskName, TaskStatus, TaskStartDate, TaskDueDate, ProjectID*

6.9.2. NF (1NF) and Converting to 2NF:

In order to satisfy 2NF, the following needs be true:

The table must be in 1NF.

- ☐ There must be no partial dependency
- ☐ Non-key attributes should not depend on a part of the composite key.

Checking for Composite Keys:

- User-1 Table: Contains a single primary key (UserID), no composite key exists.
- Project-1 Table: Composite key (ProjectID, UserID).
- Task-1 Table: Composite key (TaskID, ProjectID).

User-1 Table:

UserID $\rightarrow$ UserName, UserContact, UserEmail, UserRole

Project-1 Table:

ProjectID*, UserID* $\rightarrow$ ProjectName, ProjectStatus, ProjectStartDate, ProjectDueDate

Task-1 Table:

TaskID*, ProjectID*, UserID* $\rightarrow$ TaskName, TaskStatus, TaskStartDate, TaskDueDate, TaskPoints

Leaderboard-1 Table:

LeaderboardID*, UserID* $\rightarrow$ TotalPoints, Rank

Eliminating Partial Dependencies:

User Table (User-2):

Columns: UserID, UserName, UserContact, UserEmail, UserRole

Project Table (Project-2):

Columns: ProjectID, ProjectName, ProjectStatus, ProjectStartDate, ProjectDueDate

User_Project Table (User_Project-2):

Columns: UserID*, ProjectID*

Task Table (Task-2):

Columns: TaskID, TaskName, TaskStatus, TaskStartDate, TaskDueDate, TaskPoints

Project_Task Table (Project_Task-2):

Columns: ProjectID*, TaskID*

Leaderboard Table (Leaderboard-2):

Columns: LeaderboardID, UserID*, TotalPoints, Rank

6.9.3. NF and Converting to 3NF

In order for the database to be in 3NF it must comply with

- The database already be in 2NF.
- In other words, there cannot be any transitive dependencies, i.e., non-prime attributes cannot depend on any other non-prime attributes.

Searching for Transitive Dependencies:

In User-2 Table:

$\text{UserID} \rightarrow \text{UserName} \rightarrow X$

$\text{UserID} \rightarrow \text{UserContact} \rightarrow X$

$\text{UserContact} \rightarrow X$

- There is no transitive dependency in this table since all non-key attributes are determined by only key attributes.

In Project-2 Table:

$\text{ProjectID} \rightarrow \text{ProjectName} \rightarrow X$

$\text{ProjectID} \rightarrow \text{ProjectStatus} \rightarrow X$

$\text{ProjectID} \rightarrow \text{ProjectStartDate} \rightarrow X$

$\text{ProjectID} \rightarrow \text{ProjectDueDate} \rightarrow X$

- Non-key attributes are determined by key attributes only, which means there are no transitive dependencies in this table.

In User_Project-2 Table:

- This table does not contain more than one non-key attribute; therefore, no transitive dependencies exist.

In Project_Task-2 Table:

- There are no transitive dependencies present in this table since it does not contain multiple non-key attributes.

In Task-2 Table:

$\text{TaskID} \rightarrow \text{TaskName} \rightarrow X$

$\text{TaskID} \rightarrow \text{TaskStatus} \rightarrow X$

$\text{TaskID} \rightarrow \text{TaskStartDate} \rightarrow X$

$\text{TaskID} \rightarrow \text{TaskDueDate} \rightarrow X$

$\text{TaskID} \rightarrow \text{TaskPoints} \rightarrow X$

- Since only key attributes determine non-key attributes, there are no transitive dependencies in this table either.

In Leaderboard-2 Table:

LeaderboardID → UserID → TotalPoints, Rank

- There are no transitive dependencies here as non-key attributes depend only on the primary key.

6.9.4. Final Normalized Tables (3NF):

User-3 Table:

Columns: UserID, UserName, UserContact, UserEmail, UserRole

Project-3 Table:

Columns: ProjectID, ProjectName, ProjectStatus, ProjectStartDate, ProjectDueDate

User_Project-3 Table:

Columns: UserID*, ProjectID*

Task-3 Table:

Columns: TaskID, TaskName, TaskStatus, TaskStartDate, TaskDueDate, TaskPoints

Project_Task-3 Table:

Columns: ProjectID*, TaskID*

Leaderboard-3 Table:

Columns: LeaderboardID, UserID*, TotalPoints, Rank

Achieving 3NF

- No partial dependencies: All attributes depend on the entire primary key.
- No transitive dependencies: Non-prime attributes depend only on the primary key.
- Redundancy eliminated: The database structure is now efficient and well-normalized.

6.10. Literature Review: Task Management Systems and KPI-Driven Platforms

The development of task management systems has been greatly influenced by the incorporation of Key Performance Indicators (KPIs) and gamification techniques. These systems are designed

to improve productivity, engagement, and accountability, as demonstrated in various projects. Below is a review of similar projects, their development methodologies, and a comparison to your Sonic Points project, highlighting both similarities and differences.

1. "Research and Analysis of the Frontend Development Technologies"

- Summary: This paper discusses the essential role of HTML, CSS, and JavaScript in front-end development, emphasizing their combined use to create interactive and visually appealing web pages.
- Link: <https://www.ijeast.com/papers/47-52%2C%20Tesma611%2CIJEAST.pdf>

2. "Performance Analysis of Web Application using Microsoft .Net Technology"

- Summary: This study investigates the performance aspects of web applications developed using the Microsoft .NET Framework, highlighting its efficiency and scalability for back-end development.
- Link: https://www.researchgate.net/publication/284423419_Performance_Analysis_of_Web_Application_using_Microsoft_Net_Technology

3. "Analyzing Front-end Web Development Technologies Based on HTML, CSS, and JavaScript"

- Summary: This article provides an in-depth analysis of HTML, CSS, and JavaScript, focusing on their roles in creating responsive and user-friendly web interfaces.
- Link: <https://ijrpr.com/uploads/V5ISSUE3/IJRPR23526.pdf>

4. "Exploration of Web Front-end Development Technology and Optimization Direction"

- Summary: This paper explores various front-end development technologies, including HTML, CSS, and JavaScript, and discusses optimization strategies to enhance web performance.
- Link: https://www.researchgate.net/publication/308627965_Exploration_of_Web_front-end_development_technology_and_optimization_direction

5. "Web Development via HTML, CSS, and JavaScript"

- Summary: This study examines the use of HTML, CSS, and JavaScript in web development, providing insights into their practical applications and benefits.

- Link: <https://ijrpr.com/uploads/V5ISSUE4/IJRPR25055.pdf>

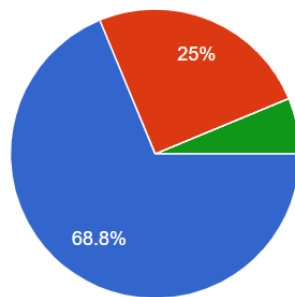
6. "Understanding Web Front-end Development Technology"

- Summary: This paper discusses the key technologies used in web front-end development, including HTML, CSS, and JavaScript, and their impact on user experience.
- Link: <https://www.irjet.net/archives/V8/i6/IRJET-V8I6597.pdf>

1. What's your biggest challenge when managing your tasks?

[Copy chart](#)

16 responses

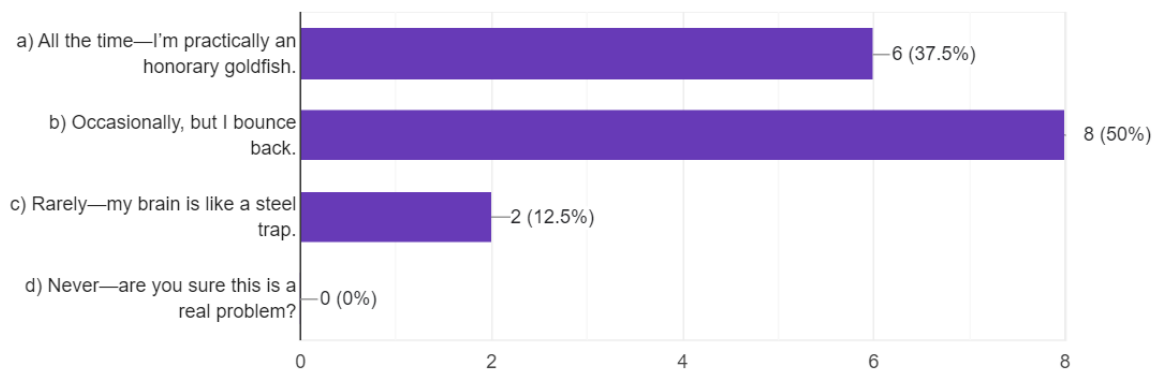


- a) My to-do list is basically a "wishful thinking" list.
- b) Motivation? Never heard of it.
- c) My tasks are wandering lost in the void.
- d) Deadlines sneak up on me like a ninja.
- e) Other (please specify)

How often do tasks just... slip your mind?

[Copy chart](#)

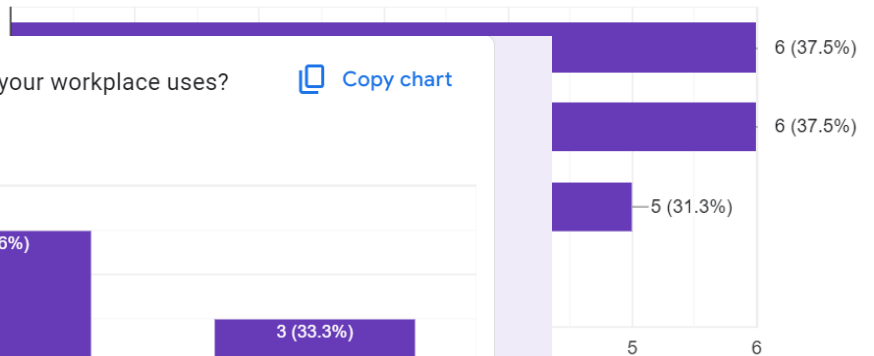
16 responses



3. How do you currently track the progress of your tasks?

 Copy chart

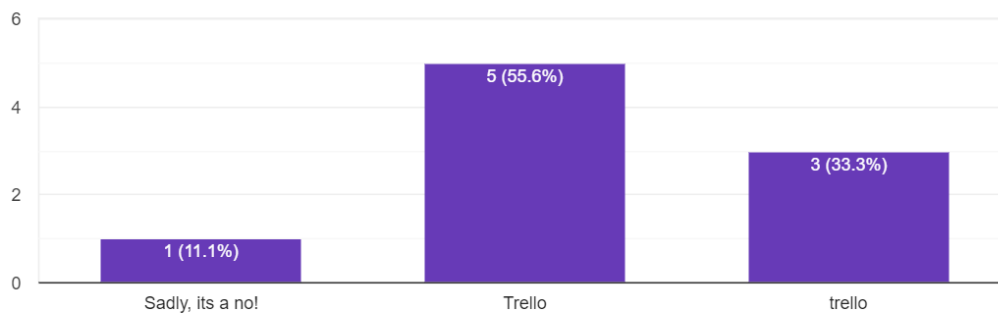
16 responses



5. If yes, what is the name of the task management tool your workplace uses?

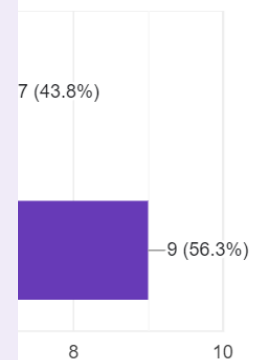
 Copy chart

9 responses



solo?

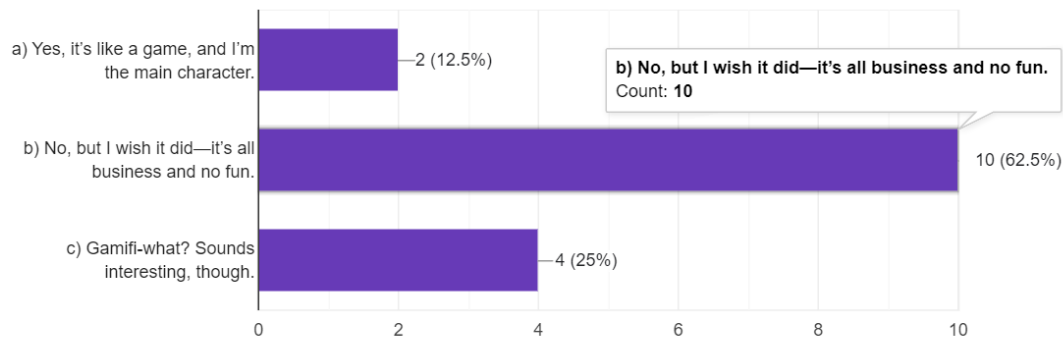
 Copy chart



6. Does your task management tool have fun gamification features like points, Rewards, or leaderboards?

 Copy chart

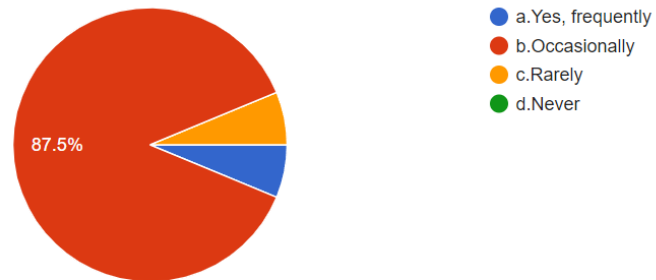
16 responses



11. Do you have trouble remembering to track completed tasks or milestones?

 [Copy chart](#)

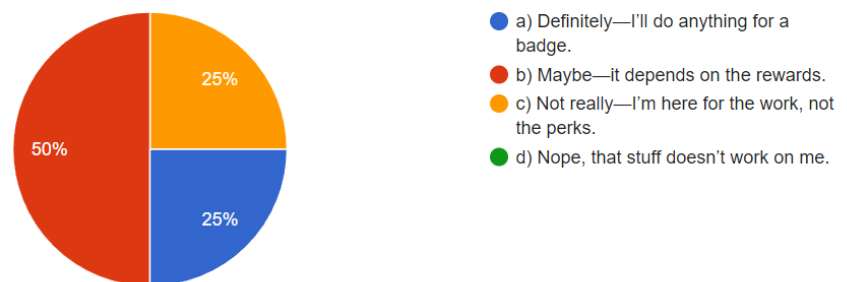
16 responses



12. Do you think rewards and competition can give your productivity a boost?

 [Copy chart](#)

16 responses



0.0 2.5 5.0 7.5 10.0 12.5

17. What's missing in your current task management tool that would make your life easier?

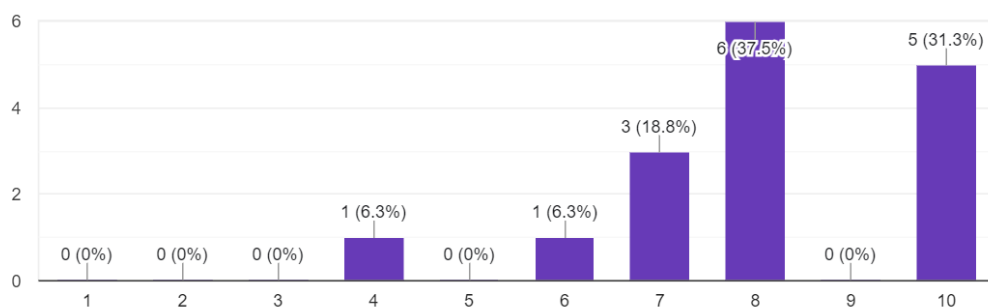
16 responses

Communication
Lack of rewards points so that i may feel like to do more task
I don't use any tools
Competitive environment
task tracking
i don't know
nothing
Reward
Actually my current task management system is missing!

7. How important are rewards and competition to keep you motivated?

[Copy chart](#)

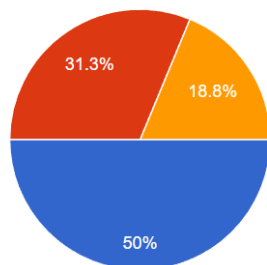
16 responses



8. Do you like seeing progress bars or other visuals for task completion?

[Copy chart](#)

16 responses

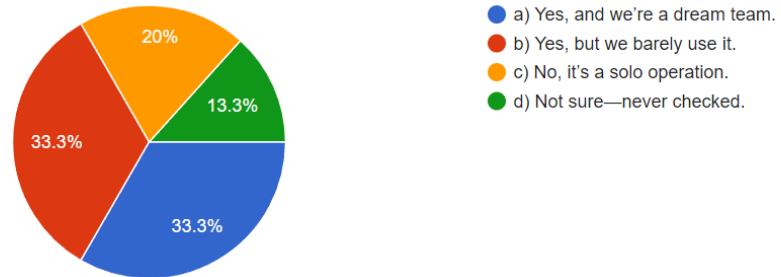


- a) Yes, it's like watching my productivity level up.
- b) Sometimes—it depends on my mood.
- c) Nah, I don't need fancy graphics to get stuff done.
- d) Progress bars? Never thought about it, but now I'm intrigued.

18. Does your tool (if you have one) let you collaborate with your team?

 [Copy chart](#)

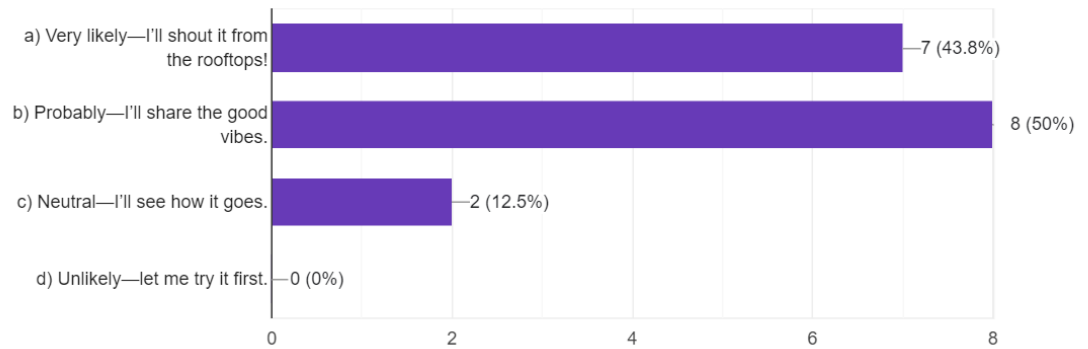
15 responses



19. How likely are you to recommend a gamified task management app to friends or co-workers?

 [Copy chart](#)

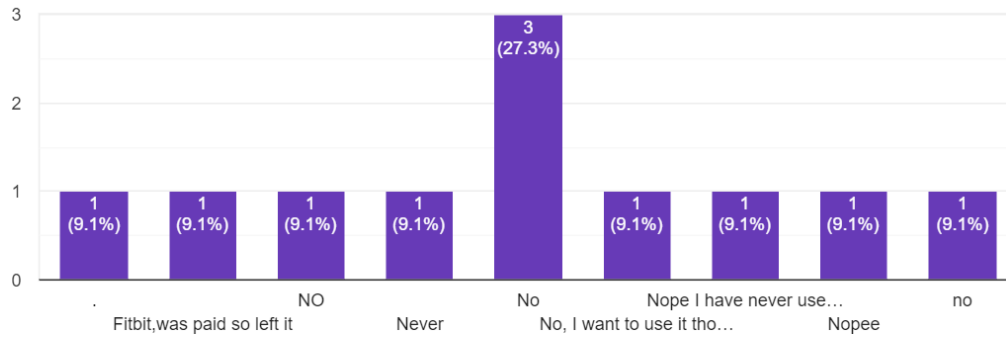
16 responses



20. Have you ever used gamified apps with task management? ? What was it like?

[Copy chart](#)

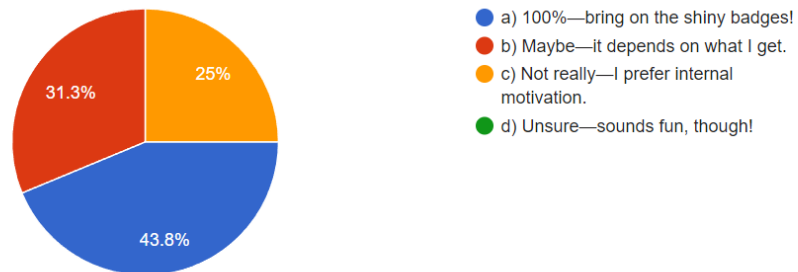
11 responses



21. If completing tasks came with rewards (like badges or leveling up), would it motivate you?

[Copy chart](#)

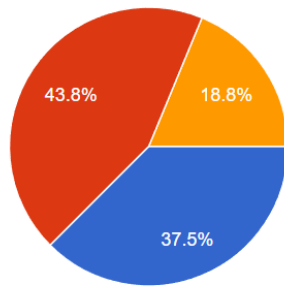
16 responses



15. Would it help if your app synced with other tools like your calendar or email?

 [Copy chart](#)

16 responses

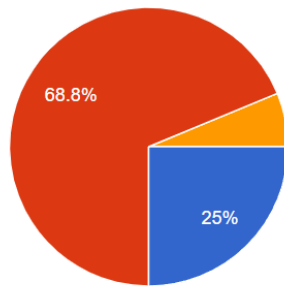


- a) Yes, please—connect ALL the things!
- b) Maybe—it depends on how well it works.
- c) Nah, I don't need that kind of integration.
- d) I'm not sure—sounds cool, though.

16. Do reminders and notifications save your day or annoy you?

 [Copy chart](#)

16 responses



- a) They're lifesavers—don't stop.
- b) They're helpful, but don't overdo it.
- c) Not helpful—I prefer to rely on my own schedule.
- d) Reminders? What reminders?